

Lecture 7: Pandas

Laboratory: Coding - Accounting, Finance and Business Consulting

Introduction to Pandas: Data Manipulation and Analysis

- **Pandas:** Built on top of NumPy, it is the go-to library for data manipulation and analysis in Python
 - Introduces data structures like Series and DataFrame (built on top of NumPy arrays) and eases data manipulation
- Name Referring to both "Panel Data", and "Python Data Analysis"
 - Created by Wes McKinney in 2008 (of the reference book "Python for Data Analysis")

Pandas: Data Manipulation Made Easy - Crucial Functionalities

- Crucial Functionalities:
 - Simplifies the process of **grouping data and applying functions to each group**
 - **Automatic alignment** of data based on labels, not just indices
 - Can manipulate measures of the same observation from different objects seamlessly
 - Pandas aligns them by index label (row name) automatically
 - Provides **methods to handle missing data easily and effectively**
 - **Plotting methods** perfectly integrate with Pandas data structures (especially Searborn)

Pandas: Data Manipulation Made Easy - More Functionalities

- More Functionalities (we only touch them):
 - Allows more than one level of indexing: **Hierarchical Indexing**
 - Time series have a hierarchical index observation-time
 - Combining datasets using merge and join operations
 - Fast and easy Input/Output functionalities: reading and writing data in different formats (CSV, Excel, SQL, etc.)

Pandas

- Central tool for data cleaning and analysis in Python

- Often used alongside NumPy, SciPy, statsmodels, scikit-learn, and seaborn/matplotlib
- Adopts NumPy's array-based computing style
 - Focuses on data processing without the need for iteration
- Pandas vs. NumPy: Key Differences
 - Pandas is designed for tabular or heterogeneous data
 - NumPy is best for homogeneously typed numerical array data
- Focuses on two main data structures: Series and DataFrame.
 - Series is a one-dimensional array-like object
 - DataFrame is a two-dimensional array-like object

Pandas Basics

Series in Pandas

- One-dimensional array-like object
 - Contains a sequence of values and an associated array of data labels called `index`
- Values must be of the same type
- Index labels can be a (possibly non-sequential) integer sequence or a string sequence
 - Index allows to reference values in the Series
 - One of the most important features of Pandas
 - Makes it easy to perform operations based on index labels
 - E.g., group operations, align data of different objects for computation or merging

Series in Pandas

```
In [1]: import numpy as np
import pandas as pd
ser1 = pd.Series([4, 7, -5, 3])
ser2 = pd.Series([4, 7, -5, 3], index=['d', 'b', 'a', 'c'])
print(f"ser1 = pd.Series([4, 7, -5, 3]) creates a Series with default index:
print(f"ser2 = pd.Series([4, 7, -5, 3], index=['d', 'b', 'a', 'c']) creates
```

`ser1 = pd.Series([4, 7, -5, 3])` creates a Series with default index:

```
0    4
1    7
2   -5
3    3
dtype: int64
```

`ser2 = pd.Series([4, 7, -5, 3], index=['d', 'b', 'a', 'c'])` creates a Series with custom index:

```
d    4
b    7
a   -5
c    3
dtype: int64
```

Index Updating

- The index of an existing Series can be modified by assignment
 - `series.index = new_index`
 - New index must have the same length as the old one
 - Usually a list of ordered values or a transformation of the old index

Index Updating

```
In [2]: print(f"ser2 is \n{ser2}")
ser2.index = ["D", "B", "A", "C"]
print(f'ser2.index = ["D", "B", "A", "C"] changes the index to uppercase: \n{ser2.index}')
ser2.index = ser2.index.str.lower()
print(f'ser2.index = ser2.index.str.lower() changes the index back to lowercase: \n{ser2.index}')
```

```
ser2 is
d    4
b    7
a   -5
c    3
dtype: int64
ser2.index = ["D", "B", "A", "C"] changes the index to uppercase:
D    4
B    7
A   -5
C    3
dtype: int64
ser2.index = ser2.index.str.lower() changes the index back to lowercase by applying the str.lower method:
d    4
b    7
a   -5
c    3
dtype: int64
```

Indexing in Pandas Series

- Allows use of labels in the index for selecting values

- Assignment of an element works as usual

```
In [3]: print(f"ser2 is: \n {ser2}")
print(f'ser2["a"] selects a single value using label: \n{ser2["a"]}')
ser2["d"] = 6
print(f'ser2["d"] = 6 updates the value at label "d": \n{ser2}')
```

```
ser2 is:
d    4
b    7
a   -5
c    3
dtype: int64
ser2["a"] selects a single value using label:
-5
ser2["d"] = 6 updates the value at label "d":
d    6
b    7
a   -5
c    3
dtype: int64
```

Indexing in Pandas Series

- You can also specify a set of values to be selected
 - Expressed as a list of strings

```
In [4]: print(f'ser2[["c", "a", "d"]] selects multiple values using a list of labels')
```

```
ser2[["c", "a", "d"]] selects multiple values using a list of labels:
c    3
a   -5
d    6
dtype: int64
```

Name Assignment to Series and Indexes

You can assign a name to the Series and its index to make it more readable

```
In [5]: ser2.name = "population"
print(f'ser3.name = "population" assigns a name to the Series: \n{ser2}')
ser2.index.name = "state"
print(f'\ser3.index.name = "state" assigns a name to the index: \n{ser2}')
```

```

ser3.name = "population" assigns a name to the Series:
d    6
b    7
a   -5
c    3
Name: population, dtype: int64
\ser3.index.name = "state" assigns a name to the index:
state
d    6
b    7
a   -5
c    3
Name: population, dtype: int64

```

Operations on Pandas Series

- Numpy Operations like filtering and Numpy Ufuncs also work on Series
 - The index-value link is unaffected by these operations

```

In [6]: print(f'ser2[ser2 > 0] filters values greater than zero: \n{ser2[ser2 > 0]}')
        print(f'np.exp(ser2) applies the exponential function: \n{np.exp(ser2)}')

```

```

ser2[ser2 > 0] filters values greater than zero:
state
d    6
b    7
c    3
Name: population, dtype: int64
np.exp(ser2) applies the exponential function:
state
d    403.428793
b   1096.633158
a     0.006738
c    20.085537
Name: population, dtype: float64

```

DataFrame in Pandas

Pandas DataFrames:

- Represent rectangular table of data
 - Contains ordered, named collection of **columns**
 - A column is a **named Series**
 - Different columns may be of different data types
 - The type of the elements in each column is the same
- Have both **row and column indexes**
 - Row indexes are called `index`
 - Column indexes are called `column`

DataFrame Creation

- Based on dictionary structure
 - Each dictionary key is a column name
 - Each dictionary value is a column of data
- Values needs to be of equal-length lists, 1D Numpy arrays, Pandas Series

DataFrame Creation

```
In [7]: dict_data = {
    "state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
    "year": [2000, 2001, 2002, 2001, 2002, 2003],
    "pop": [1500000, 1700000, 3600000, 2400000, 2900000, 3200000]
}
df1 = pd.DataFrame(dict_data)
print(f'We want to use the data in dict_data : \ndata = {dict_data}\n')
print(f'Using: df1 = pd.DataFrame(dict_data) we create a DataFrame based on
```

We want to use the data in dict_data :

```
data = {'state': ['Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada', 'Nevada'], 'year': [2000, 2001, 2002, 2001, 2002, 2003], 'pop': [1500000, 1700000, 3600000, 2400000, 2900000, 3200000]}
```

Using: df1 = pd.DataFrame(dict_data) we create a DataFrame based on the dictionary:

	state	year	pop
0	Ohio	2000	1500000
1	Ohio	2001	1700000
2	Ohio	2002	3600000
3	Nevada	2001	2400000
4	Nevada	2002	2900000
5	Nevada	2003	3200000

Pandas Data Types

- Pandas DataFrames are versatile and can hold a variety of data types (most of Numpy and more)
- Each column in a DataFrame can have its own data type
 - Use `df.dtypes` to view the data type of each column in a DataFrame
 - Use `df['column_name'].dtype` to view the data type of a single column in a DataFrame
 - Use `astype()` to convert to a given data type (the usual caveats/warnings apply)

DataFrame Index and Column Ordering

- Columns are ordered based on the key order in the dictionary
 - You can order the columns by specifying it to the `columns` parameter
 - If you pass a column that isn't in the dictionary, it will appear with missing values

- The index is automatically assigned, similar to Series
 - The DataFrame `df` will have its index starting from 0 and incrementing by 1
 - You can specify the index by passing a list to the `index` parameter or modify it using the `index` method

DataFrame Index and Column Ordering

```
In [8]: df2 = pd.DataFrame(dict_data, columns=["year", "state", "pop", "debt"])
print(f'''df2 = pd.DataFrame(dict_data, columns=["year", "state", "pop", "debt"])
Dataframe with specified column order and an additional "debt" column (with all NaN)
df2.index = ['a', 'b', 'c', 'd', 'e', 'f']# Changing the index to string labels
print(f'The command df2.index = ["a", "b", "c", "d", "e", "f"] changes the index to string labels''')
```

`df2 = pd.DataFrame(dict_data, columns=["year", "state", "pop", "debt"])` constructs a

DataFrame with specified column order and an additional "debt" column (with all NaN since no data about it is available):

	year	state	pop	debt
0	2000	Ohio	1500000	NaN
1	2001	Ohio	1700000	NaN
2	2002	Ohio	3600000	NaN
3	2001	Nevada	2400000	NaN
4	2002	Nevada	2900000	NaN
5	2003	Nevada	3200000	NaN

The command `df2.index = ["a", "b", "c", "d", "e", "f"]` changes the index to string labels:

	year	state	pop	debt
a	2000	Ohio	1500000	NaN
b	2001	Ohio	1700000	NaN
c	2002	Ohio	3600000	NaN
d	2001	Nevada	2400000	NaN
e	2002	Nevada	2900000	NaN
f	2003	Nevada	3200000	NaN

DataFrame Using a Column as Index - The `set_index` Method

- You can use the `set_index` method to create a new index based on a column
 - The column will be removed from the DataFrame
- This creates a new DataFrame object, does not modify the original one

```
In [9]: print(f"The command df2.set_index('year') creates a new dataframe using 'year' as index")
print(f"df2 is still \n{df2}\n")
```

The command `df2.set_index('year')` creates a new `dataFrame` using 'year' as the index and removes it from the columns:

	state	pop	debt
year			
2000	Ohio	1500000	NaN
2001	Ohio	1700000	NaN
2002	Ohio	3600000	NaN
2001	Nevada	2400000	NaN
2002	Nevada	2900000	NaN
2003	Nevada	3200000	NaN

`df2` is still

	year	state	pop	debt
a	2000	Ohio	1500000	NaN
b	2001	Ohio	1700000	NaN
c	2002	Ohio	3600000	NaN
d	2001	Nevada	2400000	NaN
e	2002	Nevada	2900000	NaN
f	2003	Nevada	3200000	NaN

DataFrame Using a Column as Index - The `set_index` Method - The `drop` Parameter

- The index is separate from the columns, so it is less convenient to access
 - Some variables are useful to use both as index and for computations
 - Time is often needed to identify observations, but also needed to compute observation age or distance from an event
- Use the `drop` parameter to keep the column in the DataFrame

```
In [10]: print(f"The command df2.set_index('year', drop=False)) creates a new dataFrame")
```

The command `df2.set_index('year', drop=False)` creates a new `dataFrame` using 'state' as the index and removes it from the columns:

	year	state	pop	debt
year				
2000	2000	Ohio	1500000	NaN
2001	2001	Ohio	1700000	NaN
2002	2002	Ohio	3600000	NaN
2001	2001	Nevada	2400000	NaN
2002	2002	Nevada	2900000	NaN
2003	2003	Nevada	3200000	NaN

DataFrame Using a Column as Index - The `set_index` Method - The `inplace` Parameter

- Using the parameter `inplace=True` will modify the DataFrame in place
 - **This is a functionality available in many Pandas methods**


```
In [11]: df2.set_index('year', drop=False, inplace=True)
print(f"The command df2.set_index('year', drop=False, inplace=True) updates
```

The command `df2.set_index('year', drop=False, inplace=True)` updates `df2` setting 'year' as the index and keeps it among the columns:

	year	state	pop	debt
year				
2000	2000	Ohio	1500000	NaN
2001	2001	Ohio	1700000	NaN
2002	2002	Ohio	3600000	NaN
2001	2001	Nevada	2400000	NaN
2002	2002	Nevada	2900000	NaN
2003	2003	Nevada	3200000	NaN

DataFrame Using a Column as Index - The `reset_index` Method

- The `reset_index` method does the opposite of `set_index`
 - It takes the index and makes it a column in the DataFrame
- The index is then reset to the default integer index

```
In [12]: print(f"The command df2.set_index('state').reset_index() retrieves 'state' b
df2.index = ['a', 'b', 'c', 'd', 'e', 'f']
print(f"Using df2.index = ['a', 'b', 'c', 'd', 'e', 'f'] we get to the ori
```

The command `df2.set_index('state').reset_index()` retrieves 'state' back from the index into the DataFrame:

	year	state	pop	debt
0	2000	Ohio	1500000	NaN
1	2001	Ohio	1700000	NaN
2	2002	Ohio	3600000	NaN
3	2001	Nevada	2400000	NaN
4	2002	Nevada	2900000	NaN
5	2003	Nevada	3200000	NaN

Using `df2.index = ['a', 'b', 'c', 'd', 'e', 'f']` we get to the original form of `df2` :

	year	state	pop	debt
a	2000	Ohio	1500000	NaN
b	2001	Ohio	1700000	NaN
c	2002	Ohio	3600000	NaN
d	2001	Nevada	2400000	NaN
e	2002	Nevada	2900000	NaN
f	2003	Nevada	3200000	NaN

Using Hierarchical Indexing for Multi-Dimensional Data

- Pandas dataFrames are inherently two-dimensional
 - Usually in a tidy dataset we:
 - Store different measures - in the column dimension

- For different observations - in the row dimension
- In real-world scenarios, data can be three-dimensional or higher:
 - E.g., panel data contains three dimensions: observations (row), measures (column), and time (?)
 - Numpy address this problem with 3D arrays (from MatLab)
- Hierarchical indexing provides a way to represent multi-dimensional data within the two-dimensional structure of a DataFrame
 - This feature allows to have multiple index levels on an axis
 - Capturing the complexity of panel data or higher-dimensional data

Defining a MultiIndex on Existing DataFrames

- Just pass to the `set_index` method multiple column names

```
In [13]: print(f"df1 is: \n{df1}\n")
df1.set_index(['state', 'year'], inplace=True)
print(f"The command df1.set_index(['state', 'year'], inplace=True) set a Mul
```

```
df1 is:
   state  year  pop
0  Ohio  2000  1500000
1  Ohio  2001  1700000
2  Ohio  2002  3600000
3  Nevada 2001  2400000
4  Nevada 2002  2900000
5  Nevada 2003  3200000
```

The command `df1.set_index(['state', 'year'], inplace=True)` set a MultiIndex (state, year):

```

           pop
state year
Ohio  2000  1500000
      2001  1700000
      2002  3600000
Nevada 2001  2400000
      2002  2900000
      2003  3200000
```

MultiIndexing Resources

- Multiindexing is sometimes crucial for data manipulation and analysis using external libraries
 - statsmodels, scikit-learn
- We won't cover it in detail, here you can find more resources:
 - [Python Data Science Handbook 03.05](#)
 - [MultiIndexing tutorial by Datacamp](#)

Accessing Columns in Pandas DataFrame

- Multiple ways to access columns for flexibility and convenience
- Bracket Notation
 - Use brackets and
 - Specify the column name as a string
 - Specify the column names as a list of strings
 - Works for any column name, even those with spaces or special characters

Accessing Columns in Pandas DataFrame - Bracket Notation

```
In [14]: print(f'df2["year"] accesses the "year" column using bracket notation: \n{df2["year"]}')
print(f'df2[["year", "state"]] accesses the "year" and "state" columns using bracket notation: \n{df2[["year", "state"]]}')
```

df2["year"] accesses the "year" column using bracket notation:

```
a    2000
b    2001
c    2002
d    2001
e    2002
f    2003
```

Name: year, dtype: int64

df2[["year", "state"]] accesses the "year" and "state" columns using bracket notation:

```
   year  state
a  2000   Ohio
b  2001   Ohio
c  2002   Ohio
d  2001  Nevada
e  2002  Nevada
f  2003  Nevada
```

Accessing Columns in Pandas DataFrame - Attribute/Method Notation

- Multiple ways to access columns for flexibility and convenience
- Attribute/Method notation
 - Use dot notation, similar to attribute access in Python objects
 - I.e., `df2.year` accesses the "year" column
 - Column name can be retrieved using Tab completion
 - But can retrieve just one column at time
 - Only works for valid Python variable names and non-conflicting DataFrame methods

Accessing Columns in Pandas DataFrame - Attribute/Method Notation

```
In [15]: print(f'df2.year accesses the "year" column using dot notation: \n{df2.year}')
```

df2.year accesses the "year" column using dot notation:

```
a    2000
b    2001
c    2002
d    2001
e    2002
f    2003
```

Name: year, dtype: int64

Column Creation in Pandas DataFrame

- Direct Assignment
 - Assign a scalar value or an array-like object to a new column
 - Using Lists or Arrays
 - Use a list to populate a new column. The list length must match the DataFrame length
 - Using Series
 - Index labels of the series are matched to the DataFrame's index
- Derived Columns
 - Create new columns resulting from computations on existing ones

Column Creation in Pandas DataFrame - Direct Assignment Using a Scalar Value

```
In [16]: df2["debt"] = 2e9
print(f'df2["debt"] = 2e9 adds a new column "debt" with the scalar of 2 billion')
df2["country"] = "USA"
print(f'df2["country"] = "USA" adds a new column "country" with a string value "USA"')
```

df2["debt"] = 2e9 adds a new column "debt" with the scalar of 2 billions:

	year	state	pop	debt
a	2000	Ohio	1500000	2.000000e+09
b	2001	Ohio	1700000	2.000000e+09
c	2002	Ohio	3600000	2.000000e+09
d	2001	Nevada	2400000	2.000000e+09
e	2002	Nevada	2900000	2.000000e+09
f	2003	Nevada	3200000	2.000000e+09

df2["country"] = "USA" adds a new column "country" with a string value "USA":

	year	state	pop	debt	country
a	2000	Ohio	1500000	2.000000e+09	USA
b	2001	Ohio	1700000	2.000000e+09	USA
c	2002	Ohio	3600000	2.000000e+09	USA
d	2001	Nevada	2400000	2.000000e+09	USA
e	2002	Nevada	2900000	2.000000e+09	USA
f	2003	Nevada	3200000	2.000000e+09	USA

Column Creation in Pandas DataFrame - Direct Assignment Using Lists or Arrays

- Using Lists or Arrays: Use a list to populate a new column
 - The list length must match the DataFrame length

```
In [17]: # Creating a list of values
debt_values = [1.2e9, 1.5e9, 1.7e9, 0.5e9, 0.8e9, 1.0e9]
print(f'debt_values = [1.2e9, 1.5e9, 1.7e9, 0.5e9, 0.8e9, 1.0e9] creates a l
df2["debt"] = debt_values
print(f'df2["debt"] = debt_values assigns the list to the "debt" column in f
```

debt_values = [1.2e9, 1.5e9, 1.7e9, 0.5e9, 0.8e9, 1.0e9] creates a list of v
 alues:
 [1200000000.0, 1500000000.0, 1700000000.0, 500000000.0, 800000000.0, 1000000
 000.0]

df2["debt"] = debt_values assigns the list to the "debt" column in frame2:

	year	state	pop	debt	country
a	2000	Ohio	1500000	1.200000e+09	USA
b	2001	Ohio	1700000	1.500000e+09	USA
c	2002	Ohio	3600000	1.700000e+09	USA
d	2001	Nevada	2400000	5.000000e+08	USA
e	2002	Nevada	2900000	8.000000e+08	USA
f	2003	Nevada	3200000	1.000000e+09	USA

Column Creation in Pandas DataFrame - Direct Assignment Using Series

- If you assign a Series to a DataFrame column, index labels are matched to the DataFrame's index
 - The Series index takes precedence on order in alignment
 - Any DataFrame index not present in the Series will have `NaN` in the new column

Column Creation in Pandas DataFrame - Direct Assignment Using Series - Matching Indexes

```
In [18]: # Creating a new Series with specified index that matches the DataFrame's in
income_series = pd.Series([50.2e3, 45.3e3, 60.1e3, 55.5e3, 42.8e3, 38.9e3],
print(f'""income_series = pd.Series([50.2e3, 45.3e3, 60.1e3, 55.5e3, 42.8e3,
creates a new Series with specified index:\n{income_series}\n"')
df2["income"] = income_series # Assigning the Series to a DataFrame column
print(f'df2["income"] = income_series assigns the Series to the "income" col
```

```
income_series = pd.Series([50.2e3, 45.3e3, 60.1e3, 55.5e3, 42.8e3, 38.9e3],
index=['b', 'c', 'd', 'e', 'f', 'a'])
creates a new Series with specified index:
b    50200.0
c    45300.0
d    60100.0
e    55500.0
f    42800.0
a    38900.0
dtype: float64
```

df2["income"] = income_series assigns the Series to the "income" column in df2, aligning by index:

	year	state	pop	debt	country	income
a	2000	Ohio	1500000	1.200000e+09	USA	38900.0
b	2001	Ohio	1700000	1.500000e+09	USA	50200.0
c	2002	Ohio	3600000	1.700000e+09	USA	45300.0
d	2001	Nevada	2400000	5.000000e+08	USA	60100.0
e	2002	Nevada	2900000	8.000000e+08	USA	55500.0
f	2003	Nevada	3200000	1.000000e+09	USA	42800.0

Column Creation in Pandas DataFrame - Direct Assignment Using Series - Non-Matching Indexes

```
In [19]: # Creating a new Series with specified index (not matching the one of the data frame)
val = pd.Series([2e3, 3e3, 4e3], index=['b', 'd', 'f'])
print(f"val = pd.Series([2e3, 3e3, 4e3], index=['b', 'd', 'f']) creates a new Series with specified index:
df2['income'] = val # Assigning the Series to a DataFrame column - notice that the index does not match the DataFrame index
print(f'df2["income"] = val assigns the Series to the "income" column in frame2, aligning by index:
```

```
val = pd.Series([2e3, 3e3, 4e3], index=['b', 'd', 'f']) creates a new Series with specified index:
b    2000.0
d    3000.0
f    4000.0
dtype: float64
```

df2["income"] = val assigns the Series to the "income" column in frame2, aligning by index:

	year	state	pop	debt	country	income
a	2000	Ohio	1500000	1.200000e+09	USA	NaN
b	2001	Ohio	1700000	1.500000e+09	USA	2000.0
c	2002	Ohio	3600000	1.700000e+09	USA	NaN
d	2001	Nevada	2400000	5.000000e+08	USA	3000.0
e	2002	Nevada	2900000	8.000000e+08	USA	NaN
f	2003	Nevada	3200000	1.000000e+09	USA	4000.0

Column Creation in Pandas DataFrame - Derived Columns

- Derived Columns: Create new columns based on existing ones
 - The most common occurrence**

```
In [20]: df2["per_capita_debt"] = df2["debt"] / df2["pop"]
print(f'df2["per_capita_debt"] = df2["debt"] / df2["pop"] creates a new colu
```

df2["per_capita_debt"] = df2["debt"] / df2["pop"] creates a new column "per_capita_debt" measuring per-capita debt:

	year	state	pop	debt	country	income	per_capita_debt
a	2000	Ohio	1500000	1.200000e+09	USA	NaN	800.000000
b	2001	Ohio	1700000	1.500000e+09	USA	2000.0	882.352941
c	2002	Ohio	3600000	1.700000e+09	USA	NaN	472.222222
d	2001	Nevada	2400000	5.000000e+08	USA	3000.0	208.333333
e	2002	Nevada	2900000	8.000000e+08	USA	NaN	275.862069
f	2003	Nevada	3200000	1.000000e+09	USA	4000.0	312.500000

Exercises: First Steps in Pandas

- Exercise 1: Creating a DataFrame from Named Series

You have two named Series: one containing GDP data for different countries and another containing the population data for the same countries. Create a DataFrame with these two named Series as columns. Ensure that the countries are used as the index. Compute the GDP per capita for each country and add it as a new column to the DataFrame.

```
import pandas as pd
```

```
# Sample GDP and Population data as named Series
```

```
gdp_series = pd.Series(data=[21.43e12, 14.34e12, 5.08e12, 3.86e12,
2.87e12],
                        index=['USA', 'China', 'Japan', 'Germany',
'India'],
                        name='GDP (USD)')
```

```
population_series = pd.Series(data=[331002651, 1444216107,
125580000, 83783942, 1380004385],
                              index=['USA', 'China', 'Japan',
'Germany', 'India'],
                              name='Population')
```

– Exercise 2: Trade Balance Calculation

You have a DataFrame with trade data for different countries, including imports and exports for a given year. Create a new column called "Trade Balance" that calculates the trade balance (exports – imports) for each country.

```
```python
```

```
Sample trade data
```

```
trade_data = {'Year': [2020, 2021, 2022],
 'CountryA Imports (USD)': [5000, 6000, 5500],
 'CountryA Exports (USD)': [4000, 4500, 5000],
 'CountryB Imports (USD)': [3000, 3200, 3500],
```

```
'CountryB Exports (USD)': [2800, 3000, 3300]}
```

- Exercise 3: Debt-to-GDP Ratio

You have a DataFrame with government debt and GDP data for different countries over several years. Calculate the debt-to-GDP ratio for each country-year combination and create a new column to store these ratios.

```
Sample debt and GDP data
data = {'Year': [2019, 2020, 2021, 2022],
 'CountryA Debt (USD)': [2e12, 2.2e12, 2.4e12, 2.6e12],
 'CountryA GDP (USD)': [10e12, 10.5e12, 11e12, 11.4e12],
 'CountryB Debt (USD)': [1.5e12, 1.6e12, 1.7e12, 1.8e12],
 'CountryB GDP (USD)': [8e12, 8.2e12, 8.5e12, 8.8e12]}
```

---

## Row Retrieval in Pandas DataFrame - `loc` and `iloc`

- Rows can be retrieved using `loc` and `iloc` methods
  - `loc` is label-based data selection
    - Need to specify the index label of the row to select
  - `iloc` is position-based data selection
    - Need to specify the position of the row to select

```
In [21]: print(f"df2 is: {df2}")
```

	df2 is:	year	state	pop	debt	country	income	per_capita_d
	ebt							
a	2000	Ohio	1500000	1.200000e+09	USA	NaN	800.000000	
b	2001	Ohio	1700000	1.500000e+09	USA	2000.0	882.352941	
c	2002	Ohio	3600000	1.700000e+09	USA	NaN	472.222222	
d	2001	Nevada	2400000	5.000000e+08	USA	3000.0	208.333333	
e	2002	Nevada	2900000	8.000000e+08	USA	NaN	275.862069	
f	2003	Nevada	3200000	1.000000e+09	USA	4000.0	312.500000	

## Row Retrieval in Pandas DataFrame - `loc`

- `loc` is label-based data selection
  - Need to specify the index label of the row to select

```
In [22]: print(f'df2.loc["b"] retrieves the row with index label "b": \n{df2.loc["b"]}')
print(f'df2.loc[["b", "c"]] retrieves the row with indexes label "b" and "c"')
```



```
df2.loc["b"] retrieves the row with index label "b":
year 2001
state Ohio
pop 1700000
debt 1500000000.0
country USA
income 2000.0
per_capita_debt 882.352941
Name: b, dtype: object
```

```
df2.loc[["b", "c"]] retrieves the row with indexes label "b" and "c":
 year state pop debt country income per_capita_debt
b 2001 Ohio 1700000 1.500000e+09 USA 2000.0 882.352941
c 2002 Ohio 3600000 1.700000e+09 USA NaN 472.222222
```

## Row Retrieval in Pandas DataFrame - `iloc`

- `iloc` is index-based data selection
  - Need to specify the position of the row to select

```
In [23]: print(f'df2.iloc[1] retrieves the row with integer position 2, which corresponds to label "b":')
print(f'df2.iloc[[1, 2]] retrieves the row with integer position 2 and 3, which correspond to label "b" and "c":')
```

```
df2.iloc[1] retrieves the row with integer position 2, which corresponds to label "b":
year 2001
state Ohio
pop 1700000
debt 1500000000.0
country USA
income 2000.0
per_capita_debt 882.352941
Name: b, dtype: object
```

```
df2.iloc[[1, 2]] retrieves the row with integer position 2 and 3, which correspond to label "b" and "c":
 year state pop debt country income per_capita_debt
b 2001 Ohio 1700000 1.500000e+09 USA 2000.0 882.352941
c 2002 Ohio 3600000 1.700000e+09 USA NaN 472.222222
```

## Conditional Selection in DataFrames

- Conditionals are used to filter rows based on specific criteria, same as Numpy
  - The conditional expression returns a Boolean Series

```
In [24]: # Creating a Boolean series values by comparing each column element with a scalar value
df2['year'] > 2001
print(f'df2["year"] > 2001 returns a Boolean Series: \n{df2["year"] > 2001}')
```

df2['year'] > 2001 returns a Boolean Series:

```
a False
b False
c True
d False
e True
f True
```

Name: year, dtype: bool

## Conditional Selection in DataFrames

- Conditionals are used to filter rows based on specific criteria, same as Numpy
  - The Boolean Series used in indexing selects rows with True and drops the False ones

```
In [25]: # Selecting rows where the year is greater than 2001
print(f"df2[df2['year'] > 2001] selects rows where the 'year' column is greater than 2001")
```

df2[df2['year'] > 2001] selects rows where the 'year' column is greater than 2001:

	year	state	pop	debt	country	income	per_capita_debt
c	2002	Ohio	3600000	1.700000e+09	USA	NaN	472.222222
e	2002	Nevada	2900000	8.000000e+08	USA	NaN	275.862069
f	2003	Nevada	3200000	1.000000e+09	USA	4000.0	312.500000

## Using Boolean Arrays with loc

- Boolean arrays can be used with `loc`
  - This feature is not available with `iloc`

```
In [26]: print(f"df2.loc[df2['year'] > 2001] selects rows where the 'year' column is greater than 2001")
print(f"df2.loc[df2['year'] > 2001] selects rows where the 'year' column is greater than 2001")
```

df2.loc[df2['year'] > 2001] selects rows where the 'year' column is greater than 2001:

	year	state	pop	debt	country	income	per_capita_debt
c	2002	Ohio	3600000	1.700000e+09	USA	NaN	472.222222
e	2002	Nevada	2900000	8.000000e+08	USA	NaN	275.862069
f	2003	Nevada	3200000	1.000000e+09	USA	4000.0	312.500000

## Combining Row and Column Selection in Pandas

- You can combine both row and column selection
  - In `loc`, you separate the label-based indexes of row and column with a comma
  - In `iloc`, you separate the integer-based indexing with a comma

## Combining Row and Column Selection in Pandas

```
In [27]: print(f'df2 is: \n{df2}\n')
print(f"df2.loc[['a', 'b'], ['year', 'state', 'pop']] retrieves specific rows and columns")
```

```
{df2.loc[["a", "b"], ["year", "state", "pop"]]\n''')
print(f'df2.iloc[[0, 1], [0, 1, 2]] retrieves specific rows and columns by i
```

df2 is:

	year	state	pop	debt	country	income	per_capita_debt
a	2000	Ohio	1500000	1.200000e+09	USA	NaN	800.000000
b	2001	Ohio	1700000	1.500000e+09	USA	2000.0	882.352941
c	2002	Ohio	3600000	1.700000e+09	USA	NaN	472.222222
d	2001	Nevada	2400000	5.000000e+08	USA	3000.0	208.333333
e	2002	Nevada	2900000	8.000000e+08	USA	NaN	275.862069
f	2003	Nevada	3200000	1.000000e+09	USA	4000.0	312.500000

df2.loc[["a", "b"], ["year", "state", "pop"]] retrieves specific rows and columns by label:

	year	state	pop
a	2000	Ohio	1500000
b	2001	Ohio	1700000

df2.iloc[[0, 1], [0, 1, 2]] retrieves specific rows and columns by index:

	year	state	pop
a	2000	Ohio	1500000
b	2001	Ohio	1700000

## Integer Indexing: Label-based vs Position-based

- Do not use `df[row_mask, col_mask]` without `.loc/.iloc`, you will get errors
- Chained indexing like `df[rows][cols]` can work for reading but is unsafe
- In Pandas, using integers for indexing can be ambiguous
  - The cases where is unclear if label-based or position-based indexing should be applied
- Pandas throws an error when there are ambiguities
- It is recommended to **be explicit using `loc` and `iloc`** so to avoid confusion

## Dropping Columns and Rows in Pandas - The `drop` Method

- In a data analysis workflow, you often need to drop rows and columns
  - Observations not of interest
  - Intermediate columns no longer needed
- The `drop` returns a new DataFrame without the selected rows/columns
  - The original DataFrame is not modified, but can be using `inplace=True`
- `drop` accepts various parameters, my advice is to use:
  - `columns` : column name(s) to drop
  - `index` : Index label(s) to drop

## Dropping Columns and Rows in Pandas - Dropping a Column

```
In [28]: print(f'df2 is: \n{df2}\n')
print(f'df2.drop(columns="per_capita_debt") drops the "per_capita_debt" colu
```

```
df2 is:
 year state pop debt country income per_capita_debt
a 2000 Ohio 1500000 1.200000e+09 USA NaN 800.000000
b 2001 Ohio 1700000 1.500000e+09 USA 2000.0 882.352941
c 2002 Ohio 3600000 1.700000e+09 USA NaN 472.222222
d 2001 Nevada 2400000 5.000000e+08 USA 3000.0 208.333333
e 2002 Nevada 2900000 8.000000e+08 USA NaN 275.862069
f 2003 Nevada 3200000 1.000000e+09 USA 4000.0 312.500000
```

```
df2.drop(columns="per_capita_debt") drops the "per_capita_debt" column:
```

```
 year state pop debt country income
a 2000 Ohio 1500000 1.200000e+09 USA NaN
b 2001 Ohio 1700000 1.500000e+09 USA 2000.0
c 2002 Ohio 3600000 1.700000e+09 USA NaN
d 2001 Nevada 2400000 5.000000e+08 USA 3000.0
e 2002 Nevada 2900000 8.000000e+08 USA NaN
f 2003 Nevada 3200000 1.000000e+09 USA 4000.0
```

## Dropping Columns and Rows in Pandas - Dropping a Column in Place

```
In [29]: print(f'df2 is still: \n{df2}\n')
df2.drop(columns=["country", "per_capita_debt"], inplace=True)
print(f'df2.drop(columns=["country", "per_capita_debt"], inplace=True) remov
```

```
df2 is still:
 year state pop debt country income per_capita_debt
a 2000 Ohio 1500000 1.200000e+09 USA NaN 800.000000
b 2001 Ohio 1700000 1.500000e+09 USA 2000.0 882.352941
c 2002 Ohio 3600000 1.700000e+09 USA NaN 472.222222
d 2001 Nevada 2400000 5.000000e+08 USA 3000.0 208.333333
e 2002 Nevada 2900000 8.000000e+08 USA NaN 275.862069
f 2003 Nevada 3200000 1.000000e+09 USA 4000.0 312.500000
```

```
df2.drop(columns=["country", "per_capita_debt"], inplace=True) removes from
df the the "country" and "per_capita_debt" columns:
```

```
 year state pop debt income
a 2000 Ohio 1500000 1.200000e+09 NaN
b 2001 Ohio 1700000 1.500000e+09 2000.0
c 2002 Ohio 3600000 1.700000e+09 NaN
d 2001 Nevada 2400000 5.000000e+08 3000.0
e 2002 Nevada 2900000 8.000000e+08 NaN
f 2003 Nevada 3200000 1.000000e+09 4000.0
```

## Dropping Columns and Rows in Pandas - Dropping a Row

```
In [30]: print(f'df2 is: \n{df2}\n')
print(f'df2.drop(index="a") drops the row with label "a": \n{df2.drop(index=
```

```
df2 is:
 year state pop debt income
a 2000 Ohio 1500000 1.200000e+09 NaN
b 2001 Ohio 1700000 1.500000e+09 2000.0
c 2002 Ohio 3600000 1.700000e+09 NaN
d 2001 Nevada 2400000 5.000000e+08 3000.0
e 2002 Nevada 2900000 8.000000e+08 NaN
f 2003 Nevada 3200000 1.000000e+09 4000.0
```

df2.drop(index="a") drops the row with label "a":

```
 year state pop debt income
b 2001 Ohio 1700000 1.500000e+09 2000.0
c 2002 Ohio 3600000 1.700000e+09 NaN
d 2001 Nevada 2400000 5.000000e+08 3000.0
e 2002 Nevada 2900000 8.000000e+08 NaN
f 2003 Nevada 3200000 1.000000e+09 4000.0
```

## Exercises: Pandas Data Filtering and Manipulation

- Exercise 1: Advanced Filtering and Column Selection

You have a DataFrame containing information about employees, including their names, ages, departments, and salaries. Select and display the names and ages of employees who are older than 30 and work in the "Sales" department.

```
Sample DataFrame with employee data
employee_data = {'Name': ['Alice', 'Bob', 'Charlie', 'David',
 'Eva'],
 'Age': [28, 35, 42, 29, 38],
 'Department': ['HR', 'Sales', 'Sales', 'HR',
 'Sales'],
 'Salary (USD)': [55000, 75000, 80000, 52000,
 90000]}
```

```
employee_df = pd.DataFrame(employee_data)
```

```
Exercise 1: Advanced Filtering and Column Selection
Select and display the names and ages of employees older than 30
in the "Sales" department
```

- Exercise 2: Selecting Time Periods

You have a DataFrame with monthly sales data for a retail store, including the date and sales amount. Select and display the sales data for the year 2022 only.

```
Sample DataFrame with monthly sales data
sales_data = {'Date': ['2021-12-01', '2022-01-01', '2022-02-01',
 '2022-03-01', '2022-04-01'],
 'Sales Amount (USD)': [5000, 6000, 5500, 7200, 6800]}
```

```
sales_df = pd.DataFrame(sales_data)
```

```
Exercise 2: Selecting Time Periods
```

*# Select and display the sales data for the year 2022 only*

### - Exercise 3: Unemployment Rate Trends

You have a DataFrame containing monthly unemployment rate data **for** a specific country over several years. Create a new DataFrame that shows the average unemployment rate **for** each year, **and** calculate the year **with** the highest **and** lowest average unemployment rates.

```
```python
```

```
# Sample monthly unemployment rate data
data = {'Year': [2019, 2019, 2020, 2020, 2021, 2021],
        'Month': [1, 2, 1, 2, 1, 2],
        'Unemployment Rate (%)': [4.5, 4.3, 5.2, 5.0, 6.0, 5.8]}
```

```
# Calculate the average unemployment rate for each year
# Find the year with the highest and lowest average unemployment rates
```

- Exercise 4: Selecting Time Periods

Below you have reported the dataframe `df_drop` with info on country names, populations, and per capita debt. Your task is to practice manipulating this DataFrame by using the `drop` method to remove specific columns and rows.

1. Remove the `per_capita_debt` column from `df_drop` using the `drop` method. Display the result.
2. Apply the `drop` method with `inplace=True` to remove the `country` and `per_capita_debt` columns from `df_drop`. Display `df_drop` before and after the operation.
3. Remove the row with the index label 'a' from `df_drop`. Display the result.

```
import pandas as pd
```

```
# Sample DataFrame
data = {
    'country': ['USA', 'Canada', 'Mexico', 'Germany', 'France',
               'Japan'],
    'population': [328, 37, 126, 83, 67, 126],
    'per_capita_debt': [67000, 52000, 10000, 45000, 53000, 110000]
}
df_drop = pd.DataFrame(data, index=['a', 'b', 'c', 'd', 'e', 'f'])
```

Pandas Operations and Methods

Pandas Built-in Methods

- Pandas provides a variety of built-in methods for efficient computation of common statistics and data transformations

Table 5.8: Descriptive and Summary Statistics Methods in Pandas

There are many methods for computing descriptive and summary statistics on Series and DataFrame objects

Method	Description
<code>abs()</code>	Calculates the absolute value.
<code>round()</code>	Rounds values to a specified number of decimal places.
<code>count()</code>	Number of values.
<code>value_counts()</code>	Returns a Series containing counts of unique values.
<code>describe()</code>	Generates descriptive statistics.
<code>mean()</code>	Computes the mean of the values.
<code>median()</code>	Computes the median of the values.
<code>min(), max()</code>	Finds the minimum/maximum value.
<code>idxmin(), idxmax()</code>	Compute index labels of the minimum/maximum value
<code>rank()</code>	Ranks the values within each column (different tie breaking rules can be chosen).
<code>std()</code>	Calculates the standard deviation.
<code>var()</code>	Calculates the variance.
<code>quantile()</code>	Computes the quantile of the values.
<code>corr()</code>	Computes pairwise correlation of columns.
<code>cov()</code>	Computes pairwise covariance of columns.
<code>sum()</code>	Sums up the values.
<code>prod()</code>	Calculates the product of the values.
<code>diff()</code>	Calculates the change between the current and a prior element.
<code>pct_change()</code>	Computes the percentage change between the current and a prior element.
<code>cumsum()</code>	Cumulative sum of values.
<code>cumprod()</code>	Cumulative product of values.

Method	Description
<code>cummax()</code>	Cumulative maximum of values.
<code>cummin()</code>	Cumulative minimum of values.

Pandas Built-in Methods - Column-wise and Row-wise Operations

- These functions operate column-wise by default
 - Use `axis="columns"` to apply functions across rows
 - **Examples:**
 - `df.sum()` : Sum of values in each column.
 - `df.sum(axis="columns")` : Sum of values in each row

Exploring Built-in Methods in Pandas

```
In [31]: df5 = pd.DataFrame(np.arange(12).reshape(4, 3), columns=['A', 'B', 'C'])
print(f"DataFrame df5:\n{df5}\n")
print(f"df5.sum() computes the sum of each column: \n{df5.sum()}\n")
print(f"df5.sum(axis='columns') computes the sum of each row: \n{df5.sum(axis='columns')}\n")
print(f"df5.sum(axis=1) computes the sum of each row: \n{df5.sum(axis=1)}\n")
```

DataFrame df5:

```
   A  B  C
0  0  1  2
1  3  4  5
2  6  7  8
3  9 10 11
```

`df5.sum()` computes the sum of each column:

```
A    18
B    22
C    26
dtype: int64
```

`df5.sum(axis='columns')` computes the sum of each row:

```
0     3
1    12
2    21
3    30
dtype: int64
```

`df5.sum(axis=1)` computes the sum of each row:

```
0     3
1    12
2    21
3    30
dtype: int64
```

Pandas Built-in Methods - NaN Handling

- Pandas Built-in Methods automatically handle **NaN** in computations
 - Missing values are substitute with a "reasonable" value

```
In [32]: df6 = pd.DataFrame([[1.4, np.nan], [7.1, -4.5], [np.nan, np.nan]],
                             index=["a", "b", "c"],
                             columns=["one", "two"])
print(f"df6 is: \n{df6}\n")
print(f"df6.sum(axis='columns') calculates the sum of each row: \n{df6.sum(axis='columns')}\n")
print(f"df6.mean(axis='columns') calculates the mean of each row: \n{df6.mean(axis='columns')}\n")
```

df6 is:

```
   one  two
a  1.4  NaN
b  7.1 -4.5
c  NaN  NaN
```

df6.sum(axis='columns') calculates the sum of each row:

```
   one  two
a  1.4    0
b  2.6   -4
c  0.0   -4
dtype: float64
```

df6.mean(axis='columns') calculates the mean of each row:

```
   one  two
a  1.4    0
b  1.3   -4
c  NaN   -4
dtype: float64
```

Pandas Built-in Methods - NaN Handling **skipna = False**

- Missing values exclusion can be disabled by setting **skipna = False**

```
In [33]: print(f"df6 is: \n{df6}\n")
print(f"df6.sum(axis='columns', skipna=False) calculates the sum of each row: \n{df6.sum(axis='columns', skipna=False)}\n")
print(f"df6.mean(axis='columns', skipna=False) calculates the mean of each row: \n{df6.mean(axis='columns', skipna=False)}\n")
```

```
df6 is:
   one  two
a  1.4  NaN
b  7.1 -4.5
c  NaN  NaN
```

```
df6.sum(axis='columns', skipna=False) calculates the sum of each row, including NaN:
a    NaN
b    2.6
c    NaN
dtype: float64
```

```
df6.mean(axis='columns', skipna=False) calculates the mean of each row, including NaN:
a    NaN
b    1.3
c    NaN
dtype: float64
```

Some Examples of Pandas Methods - describe

- The `describe` method computes multiple summary statistics by column
 - Using it on a DataFrame with mixed data types applies it just to numeric columns

Some Examples of Pandas Methods - describe

```
In [34]: df7 = pd.DataFrame(data = {'A': [1, 2, 3, 4], 'B': [5, 4, 3, np.nan], 'C': ['apple', 'banana', 'banana', 'banana']})
print(f"df7 is: \n{df7}\n")
print(f"df7.describe produces the following output:\n{df7.describe()}\n")
```

```
df7 is:
   A  B      C
0  1  5.0  apple
1  2  4.0  banana
2  3  3.0  banana
3  4  NaN  banana
```

df7.describe produces the following output:

```
count  4.000000  3.0
mean    2.500000  4.0
std     1.290994  1.0
min     1.000000  3.0
25%     1.750000  3.5
50%     2.500000  4.0
75%     3.250000  4.5
max     4.000000  5.0
```

Some Examples of Pandas Methods - describe

- Specifying `include = 'all'` in the call also applies `describe` to non-numeric columns adding dedicated summary statistics: Unique values, most recurring element and its frequency

```
In [35]: print(f"df7 is: \n{df7}\n")
print(f"df7.describe(include = 'all') produces the following output:\n{df7.c
```

df7 is:

	A	B	C
0	1	5.0	apple
1	2	4.0	banana
2	3	3.0	banana
3	4	NaN	banana

df7.describe(include = 'all') produces the following output:

	A	B	C
count	4.000000	3.0	4
unique	NaN	NaN	2
top	NaN	NaN	banana
freq	NaN	NaN	3
mean	2.500000	4.0	NaN
std	1.290994	1.0	NaN
min	1.000000	3.0	NaN
25%	1.750000	3.5	NaN
50%	2.500000	4.0	NaN
75%	3.250000	4.5	NaN
max	4.000000	5.0	NaN

Some Examples of Pandas Methods - `idxmax`

- `idxmax` is used to find the index of the first occurrence of the maximum value in a column.
 - Particularly useful for identifying key data points
- Example: Identifying State/Year with biggest population:

```
In [36]: print(f"df2 is: \n{df2.head()}\n")
print(f"df2['pop'].idxmax() returns the index label of the maximum value in
print(f"df2.loc[df2['pop'].idxmax(), ['state', 'year']] returns the state ar
```

```
df2 is:
   year  state      pop      debt  income
a  2000   Ohio  1500000  1.200000e+09    NaN
b  2001   Ohio  1700000  1.500000e+09  2000.0
c  2002   Ohio  3600000  1.700000e+09    NaN
d  2001  Nevada  2400000  5.000000e+08  3000.0
e  2002  Nevada  2900000  8.000000e+08    NaN
```

`df2['pop'].idxmax()` returns the index label of the maximum value in the 'pop' column:
c

`df2.loc[df2['pop'].idxmax(), ['state', 'year']]` returns the state and year with the maximum population:
state Ohio
year 2002
Name: c, dtype: object

Some Examples of Pandas Methods - `unique` and `value_counts`

- Knowing what are the unique values and their frequency is a crucial step in data cleaning and analysis
- The `unique` method returns an array of unique values in a Series
- The `value_counts` method computes a Series containing the frequency of each unique value

`unique` and `value_counts` on a Series

```
In [37]: ser6 = pd.Series(["d", "a", np.nan, "a", "b", "b", np.nan, "c"])
print(f"ser6 is: \n{ser6}\n")
print(f"ser6.unique() finds unique values: {ser6.unique()}\n")
print(f"ser6.value_counts(dropna=False) counts and the frequency of each uni
```

```
ser6 is:
0      d
1      a
2     NaN
3      a
4      b
5      b
6     NaN
7      c
dtype: object
```

ser6.unique() finds unique values: ['d' 'a' nan 'b' 'c']

ser6.value_counts(dropna=False) counts and the frequency of each unique value (including NaN):

```
a      2
NaN    2
b      2
d      1
c      1
Name: count, dtype: int64
```

unique and value_counts on a DataFrame Column

```
In [38]: print(f"df2 is: \n{df2}\n")
print(f"df2['state'].value_counts() counts and sorts the frequency of each u
print(f"df2['income'].value_counts(dropna = False) counts the frequency of e
```

```
df2 is:
   year  state   pop      debt  income
a  2000   Ohio 1500000  1.200000e+09   NaN
b  2001   Ohio 1700000  1.500000e+09  2000.0
c  2002   Ohio 3600000  1.700000e+09   NaN
d  2001  Nevada 2400000  5.000000e+08  3000.0
e  2002  Nevada 2900000  8.000000e+08   NaN
f  2003  Nevada 3200000  1.000000e+09  4000.0
```

df2['state'].value_counts() counts and sorts the frequency of each unique value in the 'state' column:

```
state
Ohio      3
Nevada    3
Name: count, dtype: int64
```

df2['income'].value_counts(dropna = False) counts the frequency of each unique value in the 'income' column (including NaN):

```
income
NaN      3
2000.0   1
3000.0   1
4000.0   1
Name: count, dtype: int64
```

Numpy Ufuncs in Pandas

- NumPy ufuncs work seamlessly with Pandas DataFrames
 - They apply element-wise to the entire DataFrame
 - Examples of ufuncs: `np.abs`, `np.sqrt`, `np.exp`, `np.log`
- Example: Using `np.abs()` to get the absolute values of DataFrame elements

Numpy Ufuncs in Pandas

```
In [39]: df5 = pd.DataFrame(np.arange(-6, 6).reshape(3, 4), columns=["a", "b", "c", "d"])
print(f"df5 is: \n{df5}\n")
print(f"np.abs(df5) applies the absolute value function to each element of df5")
```

df5 is:

	a	b	c	d
Utah	-6	-5	-4	-3
Ohio	-2	-1	0	1
Texas	2	3	4	5

`np.abs(df5)` applies the absolute value function to each element of df5:

	a	b	c	d
Utah	6	5	4	3
Ohio	2	1	0	1
Texas	2	3	4	5



Exercises: Using Pandas Built-in Methods and Ufuncs

- Exercise 1: Combining Filtering and Aggregation

You have a DataFrame with customer data, including customer names, purchase amounts, and purchase dates. Calculate and display the total purchase amount for customers who made purchases after January 1, 2022.

```
# Sample DataFrame with customer data
customer_data = {'Customer Name': ['Alice', 'Bob', 'Charlie',
                                   'David', 'Eva'],
                 'Purchase Amount (USD)': [200, 350, 120, 450,
                                           300],
                 'Purchase Date': ['2022-02-15', '2021-12-10',
                                   '2022-03-20', '2022-01-05', '2021-11-30']}
```

```
customer_df = pd.DataFrame(customer_data)
```

- Exercise 2: Financial Analysis

You have a DataFrame containing financial data for different companies. The data includes the company's stock price, market capitalization, and annual revenue. Calculate

and display the following financial ratios for each company: Price-to-Earnings ratio (P/E ratio) and Price-to-Sales ratio (P/S ratio). The formulas for these ratios are:

- P/E Ratio = Stock Price / Earnings per Share (EPS)
- P/S Ratio = Market Capitalization / Annual Revenue

```
# Sample DataFrame with financial data for different companies
financial_data = {'Company': ['Company A', 'Company B', 'Company C', 'Company D', 'Company E'],
                  'Stock Price (USD)': [45.5, 62.2, 38.9, 75.3, 56.7],
                  'Market Cap (USD)': [1200000000, 1850000000, 980000000, 2250000000, 1560000000],
                  'Annual Revenue (USD)': [450000000, 620000000, 389000000, 753000000, 567000000],
                  'Earnings per Share (EPS)': [2.5, 3.1, 2.0, 4.2, 2.8]}
```

```
financial_df = pd.DataFrame(financial_data)
```

- Exercise 3: Statistical Analysis

You have a DataFrame containing student exam scores for Math, Science, and English subjects. Compute and display the following statistics for each subject: mean, median, minimum, and maximum.

```
# Sample DataFrame with student exam scores
exam_data = {'Student': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
              'Math Score': [85, 90, 78, 92, 88],
              'Science Score': [91, 85, 89, 94, 76],
              'English Score': [82, 88, 90, 76, 84]}
```

```
exam_df = pd.DataFrame(exam_data)
```

Sorting

Sorting

- Sorted data allows to quickly identify patterns along with potential outliers/errors/missing values
 - E.g., you computed a measure of a given phenomena (mean height of children in a school) and:
 - You want to see how it varies across (groups of) observations
 - E.g., are there more females or males at the extremes?
 - Are there inexplicably tall or short children? have there been recording errors?

- Ranking is just a special case of sorting
 - Assigns ranks from one through the number of valid data points in an array
 - The rank of a value is its index in the sorted array

Sorting to Present Analysis' Results

- Apart from easing data exploration, sorting plays a major role in presenting analysis results
- One of the most common way of presenting analysis results is by means of tables
 - Pandas Dataframe are already in a tabular format
 - That are rendered as fairly nice tables in Jupyter Notebooks
 - Can also be exported in a variety of tabular formats (e.g., excel)
- A **CRUCIAL** aspect of a good presentation of result is its **clarity**
 - The information that you want to provide should be easily accessible to the reader
- Sorting is the most basic prerequisite to make a table clear and easy to read

Sorting in Pandas

- Sorting in Pandas can be done using the `sort_index` and `sort_values` methods
- `sort_index` sorts the data lexicographically by index or column label
 - By default, sorting is done in ascending order. Use `ascending=False` for descending
- `sort_values` sorts a pandas object by its values
 - Ascending takes a Boolean list to specify sorting order for each column
- Both methods return a new, sorted object
 - The original object is not modified
 - Use `inplace=True` to modify the original object

Sorting Series - `sort_index`

```
In [40]: print(f"ser2 series is: \n{ser2}\n")
print(f"ser2.sort_index() sorts by index: \n{ser2.sort_index()}\n")
print(f"ser2.sort_index(ascending=False) sorts by index in descending order:
```



```
ser2 series is:
state
d    6
b    7
a   -5
c    3
Name: population, dtype: int64
```

```
ser2.sort_index() sorts by index:
state
a   -5
b    7
c    3
d    6
Name: population, dtype: int64
```

```
ser2.sort_index(ascending=False) sorts by index in descending order:
state
d    6
c    3
b    7
a   -5
Name: population, dtype: int64
```

Sorting Series - `sort_values`

```
In [41]: print(f"ser2.sort_values() sorts by values: \n{ser2.sort_values()}\n")
        print(f"ser2.sort_values(ascending=False) sorts by values in descending order:
```

```
ser2.sort_values() sorts by values:
state
a   -5
c    3
d    6
b    7
Name: population, dtype: int64
```

```
ser2.sort_values(ascending=False) sorts by values in descending order:
state
b    7
d    6
c    3
a   -5
Name: population, dtype: int64
```

Sorting Series - the `inplace` Parameter

```
In [42]: print(f"ser2 has not been modified by previous reordering: \n{ser2}\n")
        ser2.sort_index(ascending=False, inplace=True)
        print(f"ser2.sort_index(ascending=False, inplace=True) ser2 gets changed to:
```

ser2 has not been modified by previous reordering:

state

d 6

b 7

a -5

c 3

Name: population, dtype: int64

ser2.sort_index(ascending=False, inplace=True) ser2 gets changed to:

state

d 6

c 3

b 7

a -5

Name: population, dtype: int64

Sorting DataFrames - sort_index

- A DataFrame has two indexes (rows and columns)
 - The default behaviour is sort by row index (`axis=0` / `axis='rows'`)
 - `ascending=True` sorts the index in ascending order

```
In [43]: df8 = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie', 'David'], 'Age': [32, 32, 35, 40],  
                             'Score': [85, 87, 88, 92], 'Rank': [1, 3, 2, 4]},  
                             index=['Row4', 'Row1', 'Row2', 'Row3'])  
print(f"df8 is 4x4 DataFrame of pepole traits: \n{df8}\n")  
print(f"The command df8.sort_index() sorts by row index: \n{df8.sort_index()}\n")
```

df8 is 4x4 DataFrame of pepole traits:

	Name	Age	Score	Rank
Row4	Alice	32	85	1
Row1	Bob	32	87	3
Row2	Charlie	35	88	2
Row3	David	40	92	4

The command `df8.sort_index()` sorts by row index:

	Name	Age	Score	Rank
Row1	Bob	32	87	3
Row2	Charlie	35	88	2
Row3	David	40	92	4
Row4	Alice	32	85	1

Sorting DataFrames - sort_index

```
In [44]: print(f"The parameter specification of df8.sort_index(axis='rows', ascending=True)  
print(f"The command df8.sort_index(axis='rows', ascending=False) sorts by row index")
```

The parameter specification of `df8.sort_index(axis='rows', ascending=True)` is the default behaviour:

	Name	Age	Score	Rank
Row1	Bob	32	87	3
Row2	Charlie	35	88	2
Row3	David	40	92	4
Row4	Alice	32	85	1

The command `df8.sort_index(axis='rows', ascending=False)` sorts by row index in descending order:

	Name	Age	Score	Rank
Row4	Alice	32	85	1
Row3	David	40	92	4
Row2	Charlie	35	88	2
Row1	Bob	32	87	3

Sorting DataFrames - `sort_index`

- Specifying `axis=1` or `axis=columns` sorts by column index
- Same behaviour of the `ascending` parameter

```
In [45]: print(f'''df8.sort_index(axis="columns") sorts by column index in descending
          {df8.sort_index(axis="columns")}\n''')
print(f'''df8.sort_index(axis=1, ascending=False) sorts by column index in c
{df8.sort_index(axis=1, ascending=False)}\n''')
```

`df8.sort_index(axis="columns")` sorts by column index in descending order:

	Age	Name	Rank	Score
Row4	32	Alice	1	85
Row1	32	Bob	3	87
Row2	35	Charlie	2	88
Row3	40	David	4	92

`df8.sort_index(axis=1, ascending=False)` sorts by column index in descending order:

	Score	Rank	Name	Age
Row4	85	1	Alice	32
Row1	87	3	Bob	32
Row2	88	2	Charlie	35
Row3	92	4	David	40

Sorting DataFrames - `sort_values`

- Sorting a DataFrame by values needs to specify the column to sort by
 - `by="column_name"` specifies the column to sort by
 - `by=["column_name1", "column_name2"]` specifies multiple columns to sort by

```
In [46]: df8.sort_values(['Rank'], ascending=False, inplace=True)
print(f'df8.sort_values(['Rank'], ascending=False, inplace=True) modifies df
```

```
print(f"df8.sort_values(['Age', 'Score'], ascending=[True, False]) sorts by
```

df8.sort_values(['Rank'], ascending=False, inplace=True) modifies df8 by sorting it by Rank' (descending):

	Name	Age	Score	Rank
Row3	David	40	92	4
Row1	Bob	32	87	3
Row2	Charlie	35	88	2
Row4	Alice	32	85	1

df8.sort_values(['Age', 'Score'], ascending=[True, False]) sorts by columns 'Age' (ascending) and 'Score' (descending):

	Name	Age	Score	Rank
Row1	Bob	32	87	3
Row4	Alice	32	85	1
Row2	Charlie	35	88	2
Row3	David	40	92	4

Exercises: More Exercises on Pandas Methods

You have a Series representing various market sectors with some entries being repeated and others missing. Use the Series to answer the following questions:

1. **Identify Unique Sectors:** Use the `unique` method to find all the unique market sectors in the Series.
2. **Frequency Count of Sectors:** Apply the `value_counts` method to count the frequency of each sector, excluding the missing values.
3. **Include Missing Values in Count:** Modify the previous task to include the count of missing (NaN) values using the `value_counts` method.
4. **Percentage Distribution of Sectors:** Calculate the percentage distribution of each sector (including missing values) in the Series. Ensure that the percentages sum up to 100%.

Data:

```
import pandas as pd
import numpy as np
```

```
market_sectors = pd.Series([
    "Technology", "Healthcare", "Energy", "Energy",
    "Manufacturing", np.nan, "Agriculture",
    "Technology", np.nan, "Retail", "Finance", None, "Energy",
    "Healthcare",
    "Aerospace", "Retail", "Technology", "Agriculture", "Finance",
    "Manufacturing",
    "Technology", "Energy", "Healthcare", np.nan, "Finance",
    "Retail", "Energy",
    "Agriculture", "Technology", np.nan, "Retail", "Healthcare",
```

```

None, "Manufacturing",
    "Technology", "Finance", "Retail", "Energy", "Agriculture",
"Manufacturing",
    None, "Aerospace", "Technology", "Retail", "Finance", np.nan,
"Energy",
    "Agriculture", "Manufacturing", "Retail", "Technology",
"Healthcare", "Finance"
])

```

Dealing with Missing Values and Duplicated Observations in Pandas

Missing Data

- Missing data is a common issue in data analysis
- Consists of **non-existent** or **unobserved** data
- It is important to have a clear picture of missing data in the dataset under analysis:
 - Provides info of the representativeness of the data
 - Identifies potential biases that might invalidate the analysis
 - Might point towards data collection problems

Missing Data in Pandas - Values and Methods

Pandas aims to simplify dealing with missing data

- There are two objects used as **sentinel values** for missing data:
 - **NaN** (Not a Number, from Numpy) to represent missing data for numeric values
 - **None** (from basic Python) to represent other missing data
 - This distinction is important for some operations, but these are mainly edge cases
- Pandas provides methods to handle missing data:
 - **isna** (**notna**) returns a Boolean Series indicating which values are (not) missing

Identifying and Removing Missing Data

```

In [47]: ser_nan = pd.Series([1.2, np.nan, None, -3.5, 2, 0], index=["a", "b", np.nan,
print(f"The series ser_nan is:\n{ser_nan}\n")
print(f"The command ser_nan.isna() returns a boolean Series with True for NaN v
print(f"The command ser_nan.isna().sum() easily computes the number of NaN v

```

The series `ser_nan` is:

```
a      1.2
b      NaN
NaN     NaN
d     -3.5
NaN     2.0
f       0.0
dtype: float64
```

The command `ser_nan.isna()` returns a boolean Series with True for NaN values:

```
a      False
b       True
NaN     True
d      False
NaN     False
f      False
dtype: bool
```

The command `ser_nan.isna().sum()` easily computes the number of NaN values:
2

Filtering Missing Data in Series

- To Filter Missing Data:
 - Manual filtering using `isna / notna` and Boolean indexing
 - Automated approach using `dropna` method
- `dropna` on a Series only return elements that have non-missing value
- The `inplace` parameter allows to modify the Series in place

Filtering Missing Data in Series

```
In [48]: print(f"The series ser_nan is:\n{ser_nan}\n")
print(f"The command ser_nan[ser_nan.notna()] returns it with NaN values removed")
print(f"The command ser_nan.dropna() returns it with NaN values removed: \n{ser_nan.dropna()}")
```

The series `ser_nan` is:

```
a      1.2
b      NaN
NaN     NaN
d     -3.5
NaN     2.0
f       0.0
dtype: float64
```

The command `ser_nan[ser_nan.notna()]` returns it with NaN values removed:

```
a      1.2
d     -3.5
NaN     2.0
f       0.0
dtype: float64
```

The command `ser_nan.dropna()` returns it with NaN values removed:

```
a      1.2
d     -3.5
NaN     2.0
f       0.0
dtype: float64
```

Filtering Missing Data in DataFrames

- Removing missing data in dataFrames offers more options:
 - `axis={0,1}` sets whether to drop rows or columns
 - `how={any, all}` sets whether to drop a row/column that have any/all NA
 - `thresh` sets the minimum number of non-NA values required to keep a row/column

Filtering Missing Data in DataFrames

```
In [49]: df_nan = pd.DataFrame({'A': [4, 1, 3, np.nan], 'B': [3, np.nan, 2, np.nan], 'C': [1, 2, 3, np.nan], 'D': [1, 2, 3, np.nan]})
print(f"DataFrame df_nan:\n{df_nan}\n")
print(f"The command df_nan.dropna() drops all rows with any NaN values: \n{df_nan.dropna()}\n")
```

DataFrame `df_nan`:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

The command `df_nan.dropna()` drops all rows with any NaN values:

	A	B	C	D
0	4.0	3.0	3.0	1.0

Filtering Missing Data in DataFrames

```
In [50]: print(f"DataFrame df_nan:\n{df_nan}\n")
print(f"df_nan.dropna(axis=0, how='all') drops all rows with all NaN values:")
print(f"df_nan.dropna(axis=1, thresh=3) drops all columns with less than thr
```

DataFrame df_nan:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

df_nan.dropna(axis=0, how='all') drops all rows with all NaN values:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN

df_nan.dropna(axis=1, thresh=3) drops all columns with less than three non-NaN values:

	A
0	4.0
1	1.0
2	3.0
3	NaN

Filling Missing Data in DataFrames

In some cases it is better to fill missing data instead of dropping them

- `fillna` fills missings with a specific value
 - The mean and the median are used in many applications
- Nearby (and thus possibly related) observations can be used to fill missing:
 - `bfill` uses backwards obs
 - `ffill` uses forward obs
 - `interpolate` interpolates obs
 - `limit` allows to limit the number of consecutive missing values that will be filled

Filling Missing Data in DataFrames - Fill with a Set Value:

`fillna`

```
In [51]: print(f"DataFrame df_nan:\n{df_nan}\n")
print(f"df_nan.fillna(0) fills all NaN values with 0: \n{df_nan.fillna(0)}\r")
print(f"df_nan.fillna(df_nan.mean()) fills all NaN values with the mean of t
```


DataFrame df_nan:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

df_nan.fillna(0) fills all NaN values with 0:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	0.0	0.0	2.0
2	3.0	2.0	1.0	0.0
3	0.0	0.0	0.0	0.0

df_nan.fillna(df_nan.mean()) fills all NaN values with the mean of the column:

	A	B	C	D
0	4.000000	3.0	3.0	1.0
1	1.000000	2.5	2.0	2.0
2	3.000000	2.0	1.0	1.5
3	2.666667	2.5	2.0	1.5

Filling Missing Data in DataFrames - Backward and Forward Filling: `bfill` and `ffill`

```
In [52]: print(f"DataFrame df_nan:\n{df_nan}\n")
print(f"df_nan.ffill() forward-fills the first NaN value : \n{df_nan.ffill()}\n")
print(f"df_nan.bfill() backward-fills all NaN values: \n{df_nan.bfill()}\n")
```

DataFrame df_nan:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

df_nan.ffill() forward-fills the first NaN value :

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	3.0	3.0	2.0
2	3.0	2.0	1.0	2.0
3	3.0	2.0	1.0	NaN

df_nan.bfill() backward-fills all NaN values:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	2.0	1.0	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

Filling Missing Data in DataFrames - Filling by interpolation: `interpolate`

- If there are no backward values `bfill` is used
- If there are no forward values `ffill` is used

```
In [53]: print(f"DataFrame df_nan:\n{df_nan}\n")
print(f"df_nan.interpolate() interpolates NaN values: \n{df_nan.interpolate()}\n")
```

DataFrame df_nan:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

df_nan.interpolate() interpolates NaN values:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	2.5	2.0	2.0
2	3.0	2.0	1.0	2.0
3	3.0	2.0	1.0	2.0

Filling Missing Data in DataFrames - the `axis` Parameter

- The `axis` parameter allows to specify whether to fill by column (`axis={0, rows}`, default) or by row (`axis={1, columns}`)

```
In [54]: print(f"DataFrame df_nan:\n{df_nan}\n")
print(f"df_nan.ffill(limit=1, axis='columns') forward-fills the first NaN value by row:\n{df_nan.ffill(limit=1, axis='columns')}\n")
```

DataFrame df_nan:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

df_nan.ffill(limit=1, axis='columns') forward-fills the first NaN value by row:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	1.0	NaN	2.0
2	3.0	2.0	1.0	1.0
3	NaN	NaN	NaN	NaN

Filling Missing Data in DataFrames - the `axis` Parameter

- The `axis` parameter allows to specify whether to fill by column (`axis={0, rows}`, default) or by row (`axis={1, columns}`)

```
In [55]: print(f"DataFrame df_nan:\n{df_nan}\n")
```

```
print(f"df_nan.interpolate(axis=1) interpolates NaN values by row: \n{df_nan}
```

DataFrame df_nan:

	A	B	C	D
0	4.0	3.0	3.0	1.0
1	1.0	NaN	NaN	2.0
2	3.0	2.0	1.0	NaN
3	NaN	NaN	NaN	NaN

df_nan.interpolate(axis=1) interpolates NaN values by row:

	A	B	C	D
0	4.0	3.000000	3.000000	1.0
1	1.0	1.333333	1.666667	2.0
2	3.0	2.000000	1.000000	1.0
3	NaN	NaN	NaN	NaN

Duplicate Data in Pandas

- Duplicate data is a common issue in data analysis
 - Consists of **repeated** or **identical** data
 - Notice that in duplicates both index and values are identical
- Pandas includes methods to easily detect and handle duplicate data
 - `uplicated` returns a Boolean Series indicating which rows are duplicates
 - `drop_duplicates` returns a DataFrame with duplicate rows removed
 - `drop_duplicates` has a `subset` parameter to specify which columns to consider for duplicates

Identifying and Removing Duplicates

```
In [56]: df_dup = pd.DataFrame({"col1": ["a", "b", "a", "a", "b"], "col2": [1, 1, 2,  
print(f"DataFrame df_dup:\n{df_dup}\n")  
print(f"df_dup.duplicated() returns a boolean Series with True for duplicate  
print(f"df_dup.drop_duplicates() drops duplicated rows: \n{df_dup.drop_dupli
```

DataFrame df_dup:

	col1	col2
0	a	1
1	b	1
2	a	2
3	a	1
4	b	1

df_dup.duplicated() returns a boolean Series with True for duplicated rows:

0	False
1	False
2	False
3	True
4	True

dtype: bool

df_dup.drop_duplicates() drops duplicated rows:

	col1	col2
0	a	1
1	b	1
2	a	2

Identifying and Removing Duplicates Based on a Subset of Columns

```
In [57]: df2_dup = pd.DataFrame({"col1": ["a", "b", "a", "a", "b"], "col2": [1, 1, 2, 1, 1], "col3": [0, 1, 2, 3, 4]})
print(f"DataFrame df_dup:\n{df2_dup}\n")
print(f"The command df2_dup.drop_duplicates() drops duplicated rows: \n{df2_dup.drop_duplicates()}\n")
print(f"The command df2_dup.drop_duplicates(subset=['col1', 'col2']) drops r
```

DataFrame df_dup:

	col1	col2	col3
0	a	1	0
1	b	1	1
2	a	2	2
3	a	1	3
4	b	1	4

The command df2_dup.drop_duplicates() drops duplicated rows:

	col1	col2	col3
0	a	1	0
1	b	1	1
2	a	2	2
3	a	1	3
4	b	1	4

The command df2_dup.drop_duplicates(subset=['col1', 'col2']) drops rows if the columns ['col1', 'col2'] are duplicated:

	col1	col2	col3
0	a	1	0
1	b	1	1
2	a	2	2

Duplicated Index Labels in Pandas

- Using duplicate index labels it's generally not recommended as data selection becomes messy
 - Indexing a label with multiple entries returns a Series
 - Indexing a label with a single entry returns a scalar value
- Usually happens because of a mis-specified multi-index
 - e.g., it should be obs-year but it is only obs
- **If you are getting unexpected results, check if your labels are unique**
 - The `is_unique` property can be used to check if axis labels are unique

```
In [58]: ser7 = pd.Series(np.arange(5), index=["a", "a", "b", "b", "c"])
print(f"Series with duplicate index labels: \n{ser7}\n")
print(f"ser7.index.is_unique checks for unique index labels and returns: {se
```

Series with duplicate index labels:

```
a    0
a    1
b    2
b    3
c    4
dtype: int64
```

ser7.index.is_unique checks for unique index labels and returns: False

Duplicated Index Labels in Pandas

```
In [59]: # Data selection with duplicate labels
print(f"ser7['a'] returns a Series when the index label is duplicated: \n{se
print(f"ser7['c'] returns a scalar when the index label is unique: {ser7['c'
```

ser7['a'] returns a Series when the index label is duplicated:

```
a    0
a    1
dtype: int64
```

ser7['c'] returns a scalar when the index label is unique: 4

Data Loading and Writing

Data Loading and Writing

- Data loading is a necessary initial step in (almost) any data analysis
- The main categories of Input/Output (I/O) operations are:
 1. Reading/writing text files and (more efficient) formats stored on the computer
 2. Loading/updating data from databases

- 3. Interfacing with network sources like APIs and web scraping
- We will have a quick look at the first point
- More resources on this topic: [Ch.6 of Python for Data Analysis](#)

Data Loading and Writing in Pandas

- Pandas provides several methods to handle various data formats
 - CSV, Excel, JSON, HTML, SQL, etc.

Pandas Data Loading Functions

Function	Description
<code>read_csv</code>	Load delimited data from a file, URL, or file-like object; use comma as default delimiter
<code>read_excel</code>	Read tabular data from an Excel XLS or XLSX file
<code>read_fwf</code>	Read data in fixed-width column format (i.e., no delimiters)
<code>read_pickle</code>	Read an object stored by pandas using the Python pickle format
<code>read_parquet</code>	Read the Apache Parquet binary file format
<code>read_json</code>	Read data from a JSON (JavaScript Object Notation) string representation, file, URL, or file-like object
<code>read_sas</code>	Read a SAS dataset stored in one of the SAS system's custom storage formats
<code>read_spss</code>	Read a data file created by SPSS
<code>read_sql</code>	Read the results of a SQL query (using SQLAlchemy)
<code>read_sql_table</code>	Read a whole SQL table (using SQLAlchemy); equivalent to using a query that selects everything in that table using <code>read_sql</code>
<code>read_stata</code>	Read a dataset from Stata file format
<code>read_xml</code>	Read a table of data from an XML file
<code>read_feather</code>	Read the Feather binary file format
<code>read_clipboard</code>	Variation of <code>read_csv</code> that reads data from the clipboard; useful for converting tables from web pages
<code>read_hdf</code>	Read HDF5 files written by pandas
<code>read_html</code>	Read all tables found in the given HTML document
<code>read_orc</code>	Read the Apache ORC binary file format

Handling Complex Data Encoded as Text with Pandas

- Pandas provides **many** optional arguments in data loading functions

- Real-world data is often complex and these parameters allows to handle various scenarios
 - `pandas.read_csv` has around 50 optional parameters
 - Text-encoded data is very common and text has not been designed for data storage
- The complexity of data loading function can be overwhelming
 - The [pandas documentation on data loading functions](#) provides extensive examples
 - If you face difficulties in reading a file, there is probably a relevant example that can provide insights

Optional Arguments in Data Loading Functions

- Arguments for: **Indexing and naming**
 - Allows to set one or more columns as the DataFrame index
 - There are options to get the column names from the file, from provided arguments, or use a numeric index (default)

Optional Arguments in Data Loading Functions

- Arguments for: **Type Inference and Data Conversion**
 - Purposefully designed data formats (e.g., pickle, excel) store data types, so loading is easy
 - For other formats (e.g., csv), the type of each column is not specified
 - In these cases pandas tries to guess the data type of each column
 - Usually it works but, if needed, pandas allows to specify the type by column
 - Allow to perform user-defined value conversions (flag-values)
 - Customization for handling missing value markers

Optional Arguments in Data Loading Functions

- **Date and Time Parsing**
 - Features for combining date and time information
 - Ability to merge data spread across multiple columns into a single column
- **Iterating**
 - Support for processing large files by dividing them into chunks
- **Handling Unclean Data**
 - Options for skipping rows, footers, or comments
 - Capabilities to deal with minor data issues like numeric data with comma-separated thousands

Exploring Data Loading Functions Arguments with Examples

- We will now explore some of the most common arguments in data loading functions

Some Examples of Data Loading from Text Files - ex1.csv

- Here the first example of textual data file (ex1.csv) opened with a text editor
- IT is a comma-separated-value CSV file with a header row and 3 rows of data



Setting up Our Working Environment

- **Using Relative Paths:** We'll use relative paths to access files, ensuring portability.
- **Folder Structure:** Place the `example` folder in the same location as your script.
 - Files in this folder can then be accessed using `example/filename`.

Loading csv Files - `read_csv`

```
In [60]: # reads the file marketing_rawData.csv
print(f"The command pd.read_csv('examples/ex1.csv') reads the data from ex1.csv")
pd.read_csv('examples/ex1.csv')
```

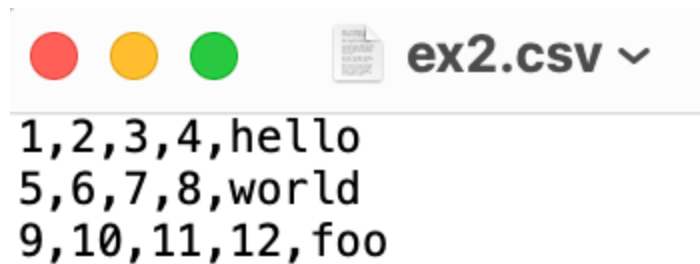
The command `pd.read_csv('examples/ex1.csv')` reads the data from `ex1.csv` using default arguments:

```
Out[60]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Loading a csv File with no Header

Sometimes data files do not have a header row



- In these cases Pandas would mistakenly use the first row as header
- The most common solutions are:
 1. Let Pandas assign default column names (integers starting at 0)
 2. Specify the column names using the `names` parameter

Loading a csv File with no Header

```
In [61]: print(f"Using pd.read_csv('examples/ex2.csv'), pandas takes the first row as  
pd.read_csv('examples/ex2.csv')
```

Using `pd.read_csv('examples/ex2.csv')`, pandas takes the first row as the header:

```
Out[61]:
```

	1	2	3	4	hello
0	5	6	7	8	world
1	9	10	11	12	foo

Loading a csv File with no Header - Default Column Names - `Header=None`

```
In [62]: print(f"Using pd.read_csv(examples/ex2.csv'), header=None), pandas sets the  
pd.read_csv('examples/ex2.csv', header=None)
```

Using `pd.read_csv(examples/ex2.csv'), header=None)`, pandas sets the default column names:

```
Out[62]:
```

	0	1	2	3	4
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Loading a csv File with no Header - Assign Column Names - `names`

```
In [63]: print(f"Using pd.read_csv('examples/ex2.csv', names = ['a', 'b', 'c', 'd', 'message'])
pd.read_csv('examples/ex2.csv', names = ['a', 'b', 'c', 'd', 'message'])
```

Using `pd.read_csv('examples/ex2.csv', names = ['a', 'b', 'c', 'd', 'message'])`
pandas sets the provided column names:

```
Out[63]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Loading a csv File - Setting a Column as Index - `index_col`

- The `index_col` parameter allows to set a column as index
 - Passing a list of column names creates a hierarchical index

```
In [64]: print(f"Using the command
pd.read_csv('examples/ex2.csv', names = ['a', 'b', 'c', 'd', 'message'], index_col = 'message')
pandas sets the provided column names and the 'message' column as index:"")
pd.read_csv('examples/ex2.csv', names = ['a', 'b', 'c', 'd', 'message'], index_col = 'message')
```

Using the command
`pd.read_csv('examples/ex2.csv', names = ['a', 'b', 'c', 'd', 'message'], index_col = 'message')`
pandas sets the provided column names and the 'message' column as index:

```
Out[64]:
```

message	a	b	c	d
hello	1	2	3	4
world	5	6	7	8
foo	9	10	11	12

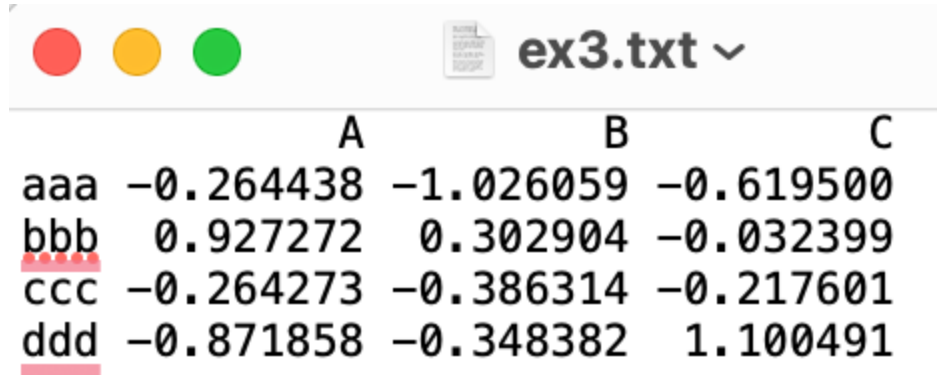
Loading a Text File with a Custom Separator - `sep`

- Some text-format data file uses a given character as delimiter (e.g., csv use commas, tsv use tabs, etc.)
 - In these cases you can use the `sep` parameter to specify the delimiter (`sep = '\t'`)
- Sep also accepts regular expressions to handle more complex cases
 - Regular expressions are a powerful tool for pattern matching
 - E.g., `\s+` matches any whitespace character (space, tab, newline, etc.),
`;` matches a semicolon or a comma
 - Regular expressions are a very powerful tool, but also very complex

- AI is very good to come up with them, but you need to know what/how to ask
- You can start learning and test RegEx here: [regex101](#)

Loading a Text File with Variable Whitespace as Delimiters

- Data in textual format may use variable whitespace



	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

- We can then use the regular expression `\s+` to handle this case

Loading a Text File with Variable Whitespace as Delimiters

```
In [65]: print(f"The command\npd.read_csv('examples/ex3.txt', sep='\s+')\nreads the c
pd.read_csv('examples/ex3.txt', sep='\s+')
```

The command

```
pd.read_csv('examples/ex3.txt', sep='\s+')
```

reads the data from ex3.txt allowing for arbitrary withspace delimitation:

```
Out [65]:
```

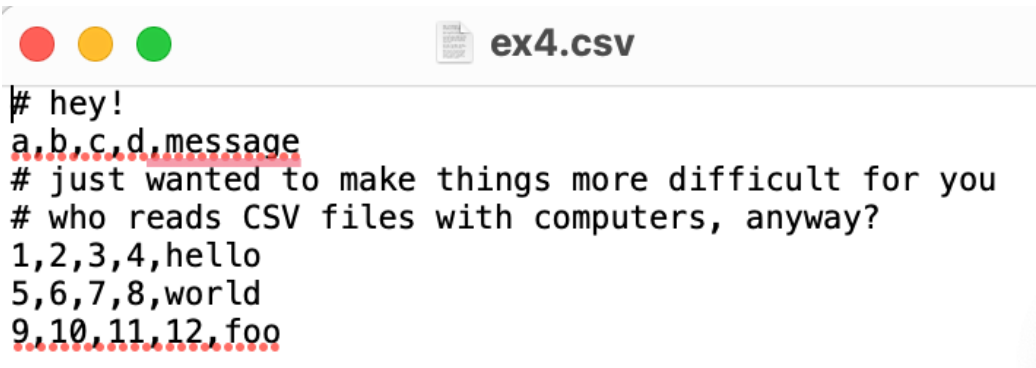
	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

- Note that the first column is now the index
 - This is done automatically by Pandas when the file has a number of data columns that is that is 1 more than the length of the header columns

Loading a Text with a Preamble or Comments - `skiprows`

- Often there are lines at the beginning of the file that are not part of the data

- E.g., a description of the data or its source



```
# hey!
a,b,c,d,message
# just wanted to make things more difficult for you
# who reads CSV files with computers, anyway?
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

- These lines can be skipped using the `skiprows` parameter

Loading a Text with a Preamble or Comments - `skiprows`

In [66]: `print(f"The command\npd.read_csv('examples/ex4.csv')\nreads the data and tries to fit it into a table (with questionable results):")`

The command
`pd.read_csv('examples/ex4.csv')`
 reads the data and tries to fit it into a table (with questionable results):

Out [66]:

					# hey!
	a	b	c	d	message
# just wanted to make things more difficult for you	NaN	NaN	NaN	NaN	NaN
# who reads CSV files with computers	anyway?	NaN	NaN	NaN	NaN
1	2	3	4	hello	
5	6	7	8	world	
9	10	11	12	foo	

Loading a Text with a Preamble or Comments - `skiprows`

In [67]: `print(f"The command\npd.read_csv('examples/ex4.csv', skiprows=[0, 2, 3])\nallows to skip the selected rows:")`

The command
`pd.read_csv('examples/ex4.csv', skiprows=[0, 2, 3])`
 allows to skip the selected rows:

Out [67]:

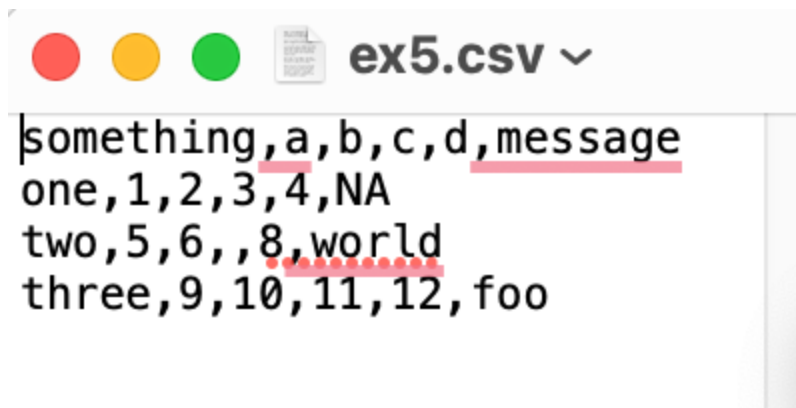
	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Handling Missing Values in File Reading

- Missing data handling is crucial in file reading
- Missing values typically appear as empty strings or sentinel values
- Pandas has a set of sentinel values that are recognized as missing values by default:
 - E.g., `NaN`, `NA`, `-1.#IND`, `1.#QNAN`, `1.#IND`, `-1.#QNAN`, `#N/A` `N/A`, `#N/A`, `N/A`, `n/a`, `NA`, `<NA>`, `#NA`, `NULL`, `null`, `NaN`, `-NaN`, `nan`, `-nan`, ``
- The `na_values` parameter allows to specify **additional** strings to be recognized as missing values
- Specifying `keep_default_na=False` allows to not consider the default sentinel values

Loading a Text with Missing Values

- Consider for example the following dataset
 - It has one NA value in the row 1 and an empty string in row 2



Loading a Text with Missing Values

```
In [68]: print(f"The command\npd.read_csv('examples/ex5.csv')\nreads the data and cor\npd.read_csv('examples/ex5.csv')
```

The command

```
pd.read_csv('examples/ex5.csv')
```

reads the data and converts to NaN the default sentinel values:

```
Out [68]:
```

	something	a	b	c	d	message
0	one	1	2	3.0	4	NaN
1	two	5	6	NaN	8	world
2	three	9	10	11.0	12	foo

Loading a Text with Missing Values - keep_default_na

```
In [69]: print(f"The command\npd.read_csv('examples/ex5.csv', keep_default_na=False)\n         pd.read_csv('examples/ex5.csv', keep_default_na=False)
```

The command

```
pd.read_csv('examples/ex5.csv', keep_default_na=False)
```

does not convert any value to NaN:

```
Out [69]:
```

	something	a	b	c	d	message
--	-----------	---	---	---	---	---------

0	one	1	2	3	4	NA
---	-----	---	---	---	---	----

1	two	5	6		8	world
---	-----	---	---	--	---	-------

2	three	9	10	11	12	foo
---	-------	---	----	----	----	-----

Loading a Text with Missing Values - keep_default_na and na_values

```
In [70]: print(f"The command\npd.read_csv('examples/ex5.csv', keep_default_na=False,\n         pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values = ['NA']))
```

The command

```
pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values = ['NA'])
```

converts to NaN only the supplied sentinel values:

```
Out [70]:
```

	something	a	b	c	d	message
--	-----------	---	---	---	---	---------

0	one	1	2	3	4	NaN
---	-----	---	---	---	---	-----

1	two	5	6		8	world
---	-----	---	---	--	---	-------

2	three	9	10	11	12	foo
---	-------	---	----	----	----	-----

Loading a Text with Missing Values - keep_default_na and na_values by Column

```
In [71]: sentinels = {"message": ["foo", "NA"], "something": ["two"]}
print(f"We define the sentinels dictionary as:\n{sentinels}\n")
print(f""The command
pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values=sentinels)
converts to NaN only the supplied sentinel values distinguishing by column:"
pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values=sentinels)
```

We define the sentinels dictionary as:

```
{'message': ['foo', 'NA'], 'something': ['two']}
```

The command

```
pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values=sentinels)
```

converts to NaN only the supplied sentinel values distinguishing by column:

```
Out[71]:
```

	something	a	b	c	d	message
0	one	1	2	3	4	NaN
1	NaN	5	6		8	world
2	three	9	10	11	12	NaN

Useful Arguments for read.csv

Argument	Description
path	String indicating filesystem location, URL, or file-like object.
sep or delimiter	Character sequence or regular expression to use to split fields in each row.
header	Row number to use as column names; defaults to 0 (first row), but should be None if there is no header row.
index_col	Column numbers or names to use as the row index in the result; can be a single name/number or a list of them for a hierarchical index.
names	List of column names for result.
skiprows	Number of rows at beginning of file to ignore or list of row numbers (starting from 0) to skip.
na_values	Sequence of values to replace with NA. They are added to the default list unless keep_default_na=False is passed.
keep_default_na	Whether to use the default NA value list or not (True by default).
comment	Character(s) to split comments off the end of lines.
parse_dates	Attempt to parse data to datetime; False by default. If True, will attempt to parse all columns. Otherwise, can specify a list of column numbers or names to parse. If element of list is tuple or list, will combine multiple columns together and parse to date.
keep_date_col	If joining columns to parse date, keep the joined columns; False by default.
converters	Dictionary containing column number or name mapping to functions (e.g., {"foo": f} would apply the function f to all values in the "foo" column).
dayfirst	When parsing potentially ambiguous dates, treat as international format (e.g., 7/6/2012 -> June 7, 2012); False by default.
date_parser	Function to use to parse dates.
nrows	Number of rows to read from beginning of file (not counting the header).
iterator	Return a TextFileReader object for reading the file piecemeal. This object can also be used with the with statement.
chunksize	For iteration, size of file chunks.
skip_footer	Number of lines to ignore at end of file.

Argument	Description
verbose	Print various parsing information, like the time spent in each stage of the file conversion and memory use information.
encoding	Text encoding (e.g., "utf-8 for UTF-8 encoded text). Defaults to "utf-8" if None.
squeeze	If the parsed data contains only one column, return a Series.
thousands	Separator for thousands (e.g., "," or "."); default is None.
decimal	Decimal separator in numbers (e.g., "." or ","); default is ".".
engine	CSV parsing and conversion engine to use; can be one of "c", "python", or "pyarrow". The default is "c", though the newer "pyarrow" engine can parse some files much faster. The "python" engine is slower but supports some features that the other engines do not.

Writing Data to Text Files

- Data can be exported to a text delimited format using `to_csv`
- Main arguments:
 - The `path_or_buf` the filepath where to write the file
 - The `sep` parameter specifies the delimiter
 - The default is the comma
 - The `index` and `header` (`True/False`) set whether to write the index and header
 - The default is to write both
 - The `na_rep` parameter specifies the string to use in place of missing values
 - The default is to write them as empty strings
 - `columns` allows to specify the columns to be saved (and their order)
 - The default is to write all columns

Writing Data to Text Files - `to_csv`

- We will be writing the dataframe of `ex5`
 1. We will load the data and apply convert NaN using a dictionary
 2. We will then save the data back

```
In [72]: df_ex5=pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values = se
print(f""""The command
df_ex5=pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values = se
converts to NaN only the supplied sentinel values distinguishing by column:"
df_ex5
```

The command

```
df_ex5=pd.read_csv('examples/ex5.csv', keep_default_na=False, na_values = se
ntinels)
```

converts to NaN only the supplied sentinel values distinguishing by column:


```
Out [72]:
```

	something	a	b	c	d	message
0	one	1	2	3	4	NaN
1	NaN	5	6		8	world
2	three	9	10	11	12	NaN

Writing Data to Text Files - `to_csv`

```
In [73]: print(f"Using the command  
df_ex5.to_csv('examples/ex5_mod.csv', sep='\t', na_rep='NaN', index=False, columns=['a', 'b', 'c'])  
the output file looks like this:")
```

Using the command

```
df_ex5.to_csv('examples/ex5_mod.csv', sep=' ', na_rep='NaN', index=False, columns=['a', 'b', 'c'])
```

the output file looks like this:

something	a	b	c	d	message
one	1	2	3	4	NaN
two	5	6		8	world
three	9	10	11	12	foo

a	b	c
1	2	3
5	6	
9	10	11

Binary Data Formats - `Pickle`

- Binary data formats are typically more efficient to read and write
 - They are designed to store data as it appears in the computer's memory
 - They are not human-readable
- Storing data in binary format can be done using Python's `pickle` module
- You can read a pickle file using `read_pickle` and write it using `to_pickle`

Saving and Loading Data in Binary Format - `to_pickle` and `read_pickle`

```
In [74]: df_ex5.to_pickle('examples/ex5_mod.pckl')  
df_ex5_frompckl = pd.read_pickle('examples/ex5_mod.pckl')  
print(f"The command df_ex5.to_pickle('examples/ex5_mod.pckl') saves the df_ex5 to a  
pckl file  
print(f"The command df_ex5_frompckl = pd.read_pickle('examples/ex5_mod.pckl') loads  
df_ex5_frompckl  
it back
```

The command `df_ex5.to_pickle('examples/ex5_mod.pckl')` saves the `df_ex5` to a `pckl` file

The command `df_ex5_frompckl = pd.read_pickle('examples/ex5_mod.pckl')` loads it back

```
Out[74]:
```

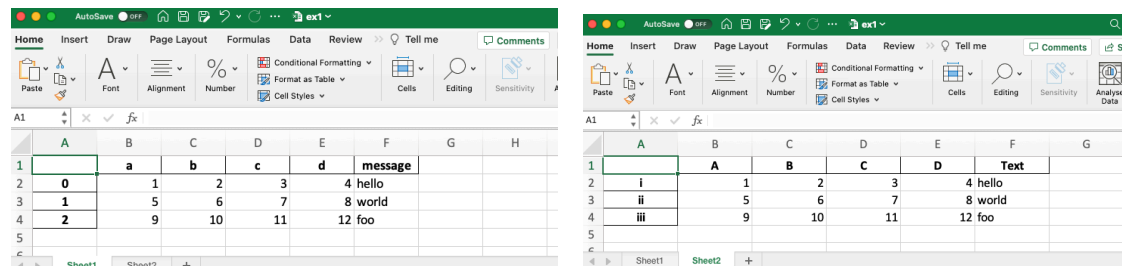
	something	a	b	c	d	message
0	one	1	2	3	4	NaN
1	NaN	5	6		8	world
2	three	9	10	11	12	NaN

Reading Microsoft Excel Files

- Pandas supports reading data from Excel 2003 and newer files
- Requires additional packages `xlrd` for XLS files and `openpyxl` for XLSX files
- The function to use is `pandas.read_excel`
 - The `sheet_name` parameter allows to specify the sheet to read
 - Can be a string (sheet name) or an integer (sheet number)
 - The default is to read the first sheet

Reading Microsoft Excel Files

Consider the following files:



Reading Microsoft Excel Files

```
In [75]: print(f"""The command
pd.read_excel('examples/ex1.xlsx', index_col=0)
reads the first sheet from ex1.xlsx: """)
pd.read_excel('examples/ex1.xlsx', index_col=0)
```

The command
`pd.read_excel('examples/ex1.xlsx', index_col=0)`
reads the first sheet from ex1.xlsx:

```
Out[75]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Reading Microsoft Excel Files

```
In [76]: print(f"""The command
pd.read_excel('examples/ex1.xlsx', sheet_name='Sheet2', index_col=0)
reads the second sheet from ex1.xlsx: """)
pd.read_excel('examples/ex1.xlsx', sheet_name='Sheet2', index_col=0)
```

The command

```
pd.read_excel('examples/ex1.xlsx', sheet_name='Sheet2', index_col=0)
reads the second sheet from ex1.xlsx:
```

```
Out[76]:
```

	A	B	C	D	Text
i	1	2	3	4	hello
ii	5	6	7	8	world
iii	9	10	11	12	foo

Iteration and Vectorization in Pandas

Iterating over Pandas DataFrames

- Iterating over Pandas DataFrames is **not recommended**
 - Pandas is optimized for vectorized operations - **iterating over rows is slow**
 - Still, iteration is the most flexible tool, so sometimes it is necessary
- You can iterate using any index method (bracket, `loc`, `iloc`)
 - The index method (`df.index`) returns a Series with the indexes for each row
 - You can use these indexes to select each row in a loop
- `iterrows` is a specifically designed method for iterating over rows
 - It returns a tuple of two elements containing the **index** and **row** values

Iterating over Pandas DataFrames - Standard Indexing

```
In [77]: df_iter = pd.DataFrame({'Student': ['Alice', 'Bob', 'Charlie', 'Ethan', 'Diana']})
print(f"The df_iter dataFrame is: \n{df_iter}\n")
for index in df_iter.index: # Loop over the index of the dataFrame
    print(f"Student at index {index} is {df_iter['Student'].iloc[index]} and")
```

The df_iter DataFrame is:

	Student	Score
0	Alice	85
1	Bob	58
2	Charlie	90
3	Ethan	45
4	Diana	73

Student at index 0 is Alice and scored 85
Student at index 1 is Bob and scored 58
Student at index 2 is Charlie and scored 90
Student at index 3 is Ethan and scored 45
Student at index 4 is Diana and scored 73

Iterating over Pandas DataFrames - iterrows

```
In [78]: print(f"The df_iter DataFrame is: \n{df_iter}\n")
for index, row in df_iter.iterrows():
    print(f"Student at index {index} is {row['Student']} and scored {row['Score']}")
```

The df_iter DataFrame is:

	Student	Score
0	Alice	85
1	Bob	58
2	Charlie	90
3	Ethan	45
4	Diana	73

Student at index 0 is Alice and scored 85
Student at index 1 is Bob and scored 58
Student at index 2 is Charlie and scored 90
Student at index 3 is Ethan and scored 45
Student at index 4 is Diana and scored 73

DataFrame's Apply Method in Pandas

- The `apply` method in Pandas is the 'intended' way to apply functions to a DataFrame
- Not needed for array statistics commonly used (e.g., `sum` and `mean`)
 - `df.sum()` and `df.mean()` are equivalent to `df.apply(sum)` and `df.apply(mean)`
- Most useful with **user-defined functions**

Advantages of Using Functions

- A function is a "chunk" of code that takes inputs (parameters) and provides an output (return value)
- In principle, it could be substituted by copy-pasting the code. **However functions allow for:**

1. **Evocative Naming:** Can be assigned an evocative name, making the code easier to understand
 2. **Centralized Code Updates:** Only need to update code in one place when needed
 3. **Reduced Error Risk:** Eliminate the chance of incidental mistakes when copy-pasting
 4. **Exploit Pandas's `apply`:** Operate on the elements of multiple rows/columns without iteration
- **Best Practices:**
 - Functions should do **only one thing** and do it well
 - Create a function if you find yourself copying and pasting the same code more than twice

Defining a Function in Python

```
def function_name(parameters, separated, by, commas):
    """docstring""" # to describe what the functions do - optional
    but advised
    body_doing_operations_on_parameters_and_storing_result
    return result
```

- `def` : Keyword to start a function definition
- `function_name` : Unique identifier for the function
- `parameters` : Variables to pass to the function
- `docstring` : Optional, for documentation of what the function does
- `body` : Code block where the function's tasks are performed
- `return` : Keyword to return a value from the function
 - if not specified the last line of the body is returned

An Example of Function Definition in Python

- **Function `pass_or_fail`:**
 - input: `score` on the test
 - output: `"Pass"` if the score is above the `passing_threshold` and `"Fail"` otherwise

```
In [79]: def pass_or_fail(score, passing_threshold):
        """Determines if the score is a pass or fail based on the passing threshold"""
        if score < 0 or score > 100:
            return "Invalid score"
        if score >= passing_threshold:
            return "PASSED"
        else:
            return "FAILED"
        print(f"pass_or_fail(score=85, passing_threshold=60) returns: {pass_or_fail(
```

```
print(f"pass_or_fail(score=50, passing_threshold=60) returns: {pass_or_fail(
print(f"pass_or_fail(score=110, passing_threshold=60) returns: {pass_or_fail(
```

```
pass_or_fail(score=85, passing_threshold=60) returns: PASSED
pass_or_fail(score=50, passing_threshold=60) returns: FAILED
pass_or_fail(score=110, passing_threshold=60) returns: Invalid score
```

apply a Custom Function on a Series

Let's use the function `pass_or_fail` to create a new column identifying which students passed the test

```
In [80]: print(f"The df_iter DataFrame is: \n{df_iter}\n")
print(f"Running df_iter['Result'] = df_iter['Score'].apply(pass_or_fail, pas
df_iter['Result'] = df_iter['Score'].apply(pass_or_fail, passing_threshold=5
print(f"The exam_results becomes: \n{df_iter}\n")
```

The df_iter DataFrame is:

	Student	Score
0	Alice	85
1	Bob	58
2	Charlie	90
3	Ethan	45
4	Diana	73

Running `df_iter['Result'] = df_iter['Score'].apply(pass_or_fail, passing_threshold=55)`, creates a new column with the result of the function on the Score column

The exam_results becomes:

	Student	Score	Result
0	Alice	85	PASSED
1	Bob	58	PASSED
2	Charlie	90	PASSED
3	Ethan	45	FAILED
4	Diana	73	PASSED

map: a Limited but Faster apply

- `apply` is **flexible**: can operate on multiple DataFrame columns and the function can take multiple arguments
- `map` is a simpler method that:
 - Only operates on **Series**
 - Only accepts single-argument functions
 - Is typically more efficient
- Use cases
 - Use `map` for single column transformations
 - Opt for `apply` in more elaborate cases

map a Limited but Faster apply

- Let's define a simple function that extracts the last 3 characters of a string

```
In [81]: def lastThreeChars(x):  
        """Returns the last three characters of a string."""  
        return x[-3:]  
print(f"Running df2['state'].map(lastThreeChars) returns: \n{df2['state'].ma
```

Running df2['state'].map(lastThreeChars) returns:

```
a    hio  
b    hio  
c    hio  
d    ada  
e    ada  
f    ada
```

Name: state, dtype: object

apply a Custom Function on a DataFrame - Using Multiple Columns

- apply allows to define functions that combine multiple columns
- Let's define a function that returns the ratio of two columns

```
In [82]: print(f"The function calculate_debt_to_income(row), calculates the ratio of  
def calculate_debt_to_income(row):  
    if row['income'] != 0:  
        return row['debt'] / row['income']  
    else:  
        return np.nan
```

The function calculate_debt_to_income(row), calculates the ratio of the columns 'debt' and 'income'

apply a Custom Function on a DataFrame - Using Multiple Columns

```
In [83]: print(f"df2 is:\n{df2}\n")  
df2['debt_to_income'] = df2.apply(calculate_debt_to_income, axis=1)  
print(f"df2['debt_to_income'] = df2.apply(calculate_debt_to_income, axis=1)
```

```
df2 is:
   year  state      pop      debt  income
a  2000   Ohio  1500000  1.200000e+09    NaN
b  2001   Ohio  1700000  1.500000e+09  2000.0
c  2002   Ohio  3600000  1.700000e+09    NaN
d  2001  Nevada  2400000  5.000000e+08  3000.0
e  2002  Nevada  2900000  8.000000e+08    NaN
f  2003  Nevada  3200000  1.000000e+09  4000.0
```

df2['debt_to_income'] = df2.apply(calculate_debt_to_income, axis=1) creates a new column 'debt_to_income' in df2:

```
   year  state      pop      debt  income  debt_to_income
a  2000   Ohio  1500000  1.200000e+09    NaN            NaN
b  2001   Ohio  1700000  1.500000e+09  2000.0    750000.000000
c  2002   Ohio  3600000  1.700000e+09    NaN            NaN
d  2001  Nevada  2400000  5.000000e+08  3000.0    166666.666667
e  2002  Nevada  2900000  8.000000e+08    NaN            NaN
f  2003  Nevada  3200000  1.000000e+09  4000.0    250000.000000
```

Lambda Functions in Python: Simplifying Short Operations

- Sometimes you only need to do an operation once
 - Defining a function in these cases is overkill
- Lambda functions are **anonymous functions** that can be defined in a single line
 - Ideal for simple, temporary functions
- Syntax:

lambda parameters_separated_by_commas:
body_doing_operations_on_parameters

- Consists of the **lambda** keyword, followed by **the parameters, a colon, and an expression**
 - Can have any number of parameters but only one expression
 - The expression is evaluated and returned

Lambda Functions in Python: Simplifying Short Operations - An Example

```
In [84]: print(f"Running df2['state'].apply(lambda x: x[-3:]) returns: \n{df2['state']
```

```
Running df2['state'].apply(lambda x: x[-3:]) returns:
```

```
a   hio
b   hio
c   hio
d   ada
e   ada
f   ada
```

```
Name: state, dtype: object
```



Exercise: Functions, Apply, Map, Sorting, and Ranking

- Exercise 1: Salary Category

1. Write a function named `salary_category` that categorizes salaries into 'Low', 'Medium', and 'High'. Use thresholds at 50,000 and \$70,000 for the categories.
2. Apply this function to the 'Salary' column of a DataFrame that contains 'Age' and 'Salary' of individuals.
3. Add a new column to the DataFrame called 'Category' which shows the salary categories for each row. Display the updated DataFrame.

```
data1 = pd.DataFrame({  
    'Age': [25, 30, 45, 28, 35],  
    'Salary': [50000, 55000, 80000, 45000, 70000]  
})
```

- Exercise 2: Product Quality Assessment

1. Create a function `quality_assessment` that takes a score (0-100) and returns 'Poor', 'Average', 'Good', or 'Excellent' based on two thresholds to be provided as arguments.
2. Apply this function to each row of a DataFrame that contains product names and their quality scores.
3. Add a new column 'Quality' to the DataFrame that displays the quality assessment for each product. Show the updated DataFrame.

```
data2 = pd.DataFrame({  
    'Product': ['Widget', 'Gadget', 'Thingamajig', 'Doohickey',  
    'Gizmo'],  
    'Quality_Score': [85, 58, 75, 45, 93]  
})
```

- Exercise 3: Product Pricing Analysis

1. Use lambda functions to ensure the 'Price' column is of float type and the 'Quantity' column is of integer type in a DataFrame that contains products, their prices, quantities, and discount availability.
2. Write a lambda function to convert the 'Discount' column to boolean values (True for 'Yes', False for 'No').
3. Create a function `total_sales` that calculates the total sales (price * quantity) and applies a 10% discount if the discount is True.
4. Apply `total_sales` to each row of the DataFrame and add a new column 'Total_Sales'. Display the DataFrame with this new column.

```
data3 = pd.DataFrame({  
    'Product': ['Apple', 'Banana', 'Carrot', 'Date', 'Eggplant'],  
    'Price': [1.5, 0.5, 2.0, 3.0, 4.5],  
    'Quantity': [10, 20, 15, 5, 7],  
    'Discount': ['Yes', 'No', 'Yes', 'No', 'Yes']  
})
```

- Exercise 4: Trimmed Mean Comparison Across Products

1. Write a function named `trimmed_mean` that calculates the trimmed mean of a list of numbers. The function should take two parameters: the list of numbers and the number of values to trim from each end. For example, `trim = 5` would remove the lowest 5 and the highest 5 values.
2. Apply this function to the columns of a DataFrame, each representing the sales figures for different products over a period. Compare the trimmed means to evaluate the performance consistency of the products.
3. Order the columns of the DataFrame by their trimmed means in ascending order. Display the DataFrame with the ordered columns.

```
data4 = pd.DataFrame({
    'Product_A': [120, 110, 130, 115, 125, 140, 135, 150, 145,
155],
    'Product_B': [200, 210, 190, 205, 195, 180, 185, 175, 170,
160],
    'Product_C': [300, 310, 320, 305, 295, 285, 275, 265, 255, 245]
})
```

- Exercise 5: Advanced Facility Efficiency Analysis

1. Create a function named `efficiency_score` that computes an efficiency score for each facility. The score should be based on a combination of parameters like energy usage, output, and operational hours. The function should take these parameters as input and return a calculated score. You are left to decide the exact formula for the score, make it relevant!
2. Apply this function to each row of a DataFrame where each row represents different facilities with their respective energy usage, output, and operational hours.
3. Add a new column 'Efficiency_Score' to the DataFrame that contains the computed scores. Use these scores to show (order) which facility is operating most efficiently.

```
data5 = pd.DataFrame({
    'Facility': ['Alpha', 'Beta', 'Gamma', 'Delta', 'Epsilon'],
    'Energy_Usage': [2000, 1800, 2200, 2100, 1900], # Energy usage
in kWh
    'Output': [400, 350, 450, 425, 375], # Output in units
    'Operational_Hours': [50, 48, 55, 52, 49] # Hours per week
})
```

Group Operations in Pandas

Group Operations

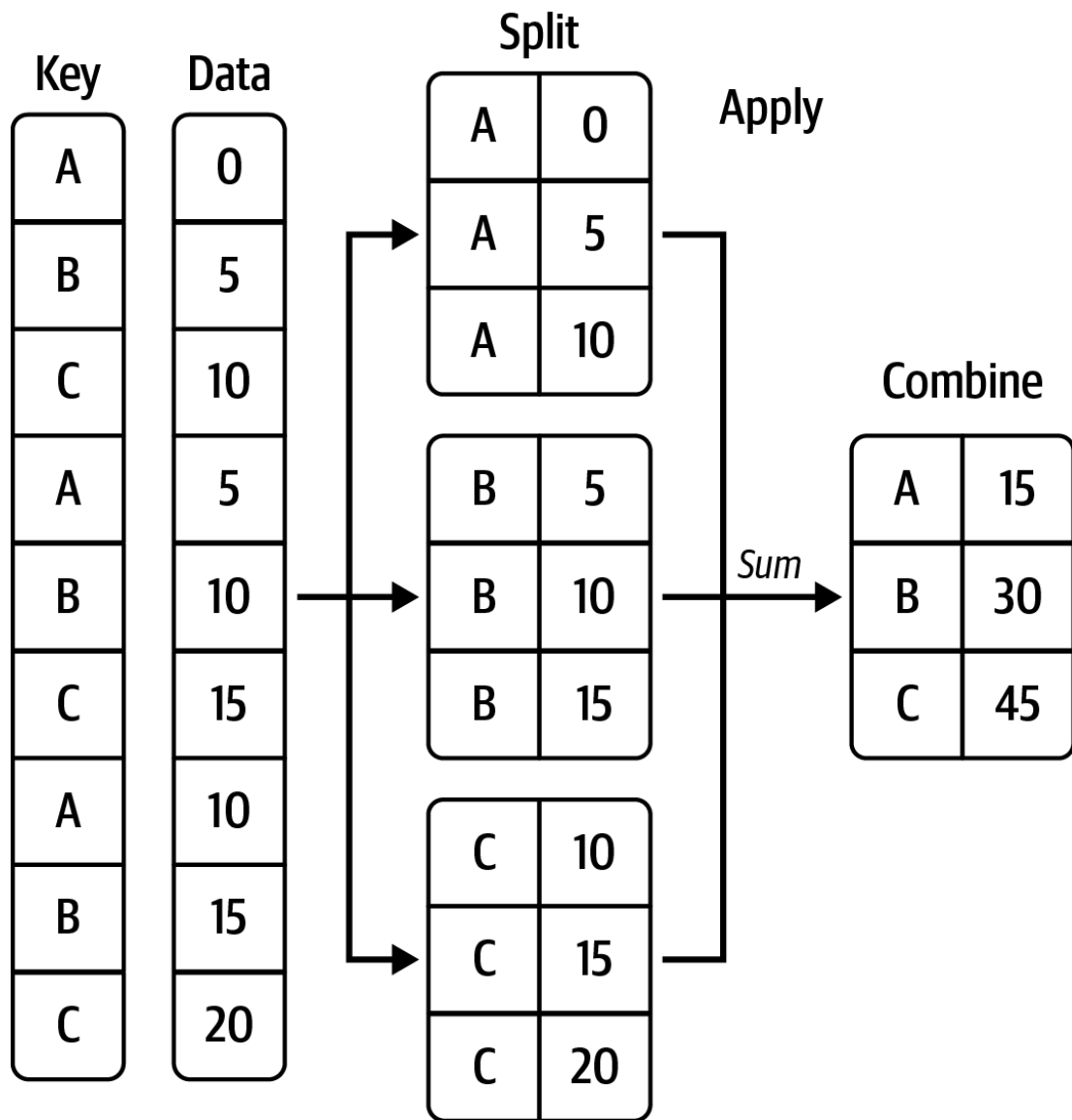
- Group operations:

- **Split** a dataset into groups
- **Apply** a function to each group independently
- **Combine** the results into a data structure
- Computing how a measure varies across groups:
 - Allow to characterize the measure (its distribution, its mean) for each group
 - E.g., how the height of children varies across schools
 - Allows to see how the measure varies across groups → investigate its relationship with the group variable
 - E.g., how the height of children varies with exposure to pollutants

Understanding the Split-Apply-Combine Strategy in pandas

- **A Three-Stage Process:**
 1. **Splitting:**
 - Dividing the data (Pandas DataFrame) into groups based on one or more keys
 2. **Applying:**
 - Performing a function on each group
 - This will create a new object that might be:
 - Scalar (resulting from an aggregation, e.g., count, sum)
 - Composite (e.g., DataFrame or Series)
 3. **Combining:**
 - Aggregating the results from each group into a single result object
 - The structure of the result depends on the operation performed

Split-Apply-Combine - An Example Splitting by Key and Applying sum



Grouping Keys

- Grouping keys can vary in type:
 - Column Names:** Of the DataFrame to be grouped (most common)
 - List/Array:** Directly specifying the groups (the length must match with the axis to group)
 - Dictionary/Series:** Groups together obs whose index is associated to the same value in the Dictionary/Series
 - Index Levels:** Groups together obs with the same value in a given level of a multiindex
 - Function:** Groups together obs with the same `function(index)` result

An Example Dataset to Group

```
In [85]: dfGroup = pd.DataFrame({ "key1" : ["a", "a", "a", None, "b", "b", None],
                                   "key2" : pd.Series([1, 2, 1, 2, 1, None, 1], dtype="Int64")
```

```

        "data1" : np.arange(0,7),
        "data2" : np.arange(10,17)})
print(f"dfGroup is: \n{dfGroup}\n")

```

```

dfGroup is:
   key1  key2  data1  data2
0     a     1      0     10
1     a     2      1     11
2     a     1      2     12
3  None     2      3     13
4     b     1      4     14
5     b  <NA>      5     15
6  None     1      6     16

```

Grouping using `groupby`

- The `groupby` is the method to group rows in a DataFrame
 - This creates a GroupBy object (think of it as a collection of DataFrames, one for each group)
 - One of the most frequent keys used is a column name
 - E.g., `grouped_oneKey = dfToGroup.groupby("key1")`
 - It's an intermediate step augmenting the data with the group info
- We can then define an aggregation method on the grouped object
 - We might want to calculate group means
 - We can use the `mean` method on the DataFrameGroupBy object
 - `grouped_oneKey.mean()` computes the mean for each group based on the key variable

An Example Dataset to Group

```

In [86]: dfToGroup = pd.DataFrame({ "key1" : ["a", "a", "a", None, "b", "b", None],
                                   "key2" : pd.Series([1, 2, 1, 2, 1, None, 1], dtype="Int64"),
                                   "data1" : np.arange(0,7),
                                   "data2" : np.arange(10,17)})
print(f"dfToGroup is: \n{dfToGroup}\n")

```

```

dfToGroup is:
   key1  key2  data1  data2
0     a     1      0     10
1     a     2      1     11
2     a     1      2     12
3  None     2      3     13
4     b     1      4     14
5     b  <NA>      5     15
6  None     1      6     16

```

Grouping - Using a Column Name as Key

```
In [87]: dfGrouped_oneKey = dfToGroup.groupby("key1")
print(f"The object dfGrouped_oneKey, defined as dfGroup.groupby('key1') is:
print(f"The command dfGrouped_oneKey.mean() returns: \n{dfGrouped_oneKey.mean()}"
```

The object dfGrouped_oneKey, defined as dfGroup.groupby('key1') is: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x11294c220>

The command dfGrouped_oneKey.mean() returns:

	key2	data1	data2
key1			
a	1.333333	1.0	11.0
b	1.0	4.5	14.5

Iterating over Groups - Understanding the GroupBy object

- The GroupBy object is a collection of DataFrames, one for each group
- If you use it as an iterable it returns a tuple with:
 - The group key
 - The group DataFrame
- Great for exploring the data and debugging (and understanding the groupby operation)

Iterating over Groups - Understanding the GroupBy object

```
In [88]: for name, group in dfGrouped_oneKey:
print(f"Group name: {name}")
print(f"Group dataframe:\n{group}\n")
```

Group name: a
Group dataframe:

	key1	key2	data1	data2
0	a	1	0	10
1	a	2	1	11
2	a	1	2	12

Group name: b
Group dataframe:

	key1	key2	data1	data2
4	b	1	4	14
5	b	<NA>	5	15

- The dataframe has been split into two groups based on the values of the key variable
- Each column of each group is aggregated using the `mean` method
- The result is a dataframe with the mean of each column for each group

Grouping - Using Multiple Column Names as Key

```
In [89]: dfGrouped_twoKeys = dfToGroup.groupby(['key1', 'key2'])
print(f"The object dfGrouped_twoKeys, defined as dfToGroup.groupby(['key1',
print(f"The command dfGrouped_twoKeys.mean() returns: \n{dfGrouped_twoKeys.m
```

The object dfGrouped_twoKeys, defined as dfToGroup.groupby(['key1', 'key2']) is: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x11294c520>

The command dfGrouped_twoKeys.mean() returns:

		data1	data2
key1	key2		
a	1	1.0	11.0
	2	1.0	11.0
b	1	4.0	14.0

- Grouping by two or more keys creates groups based on all the combinations of their values
 - Hierarchical indexing

Grouping - Using arrays/Lists as Key

```
In [90]: key_states = np.array(["OH", "CA", "CA", "OH", "OH", "CA", "OH"])
key_years = [2005, 2005, 2006, 2005, 2006, 2005, 2006]
print(f"The array 'key_states' and the List 'years' are:\n{key_states}\n{key
dfGrouped_twoKeys_al = dfToGroup.groupby([key_states, key_years])
print(f"These objects can be used as keys on dfToGroup using grouped_twoKeys
print(f"The command grouped_twoKeys_al.mean(numeric_only=True) returns:\n{df
```

The array 'key_states' and the List 'years' are:

```
['OH' 'CA' 'CA' 'OH' 'OH' 'CA' 'OH']
[2005, 2005, 2006, 2005, 2006, 2005, 2006]
```

These objects can be used as keys on dfToGroup using grouped_twoKeys_al = dfToGroup.groupby([key_states, key_years])

The command grouped_twoKeys_al.mean(numeric_only=True) returns:

		key2	data1	data2
CA	2005	2.0	3.0	13.0
	2006	1.0	2.0	12.0
OH	2005	1.5	1.5	11.5
	2006	1.0	5.0	15.0

- Notice that we added the option `numeric_only=True` to avoid errors for computing the mean of the non-numeric column 'key1'

Grouping - Using Series/Dictionaries as Key

- Using a Series/Dictionary as key maps the index of the DataFrame to the associated value

```
In [91]: key_plants = pd.Series(["Plant A", "Plant A", "Plant B", "Plant B", "Plant C", "Plant C", "Plant C"])
print(f"The plants series is: \n{key_plants}\n")
print(f"Remember that dfToGroup is: \n{dfToGroup}\n")
```

The plants series is:

```
0    Plant A
1    Plant A
2    Plant B
3    Plant B
4    Plant C
5    Plant C
6    Plant C
```

dtype: object

Remember that dfToGroup is:

	key1	key2	data1	data2
0	a	1	0	10
1	a	2	1	11
2	a	1	2	12
3	None	2	3	13
4	b	1	4	14
5	b	<NA>	5	15
6	None	1	6	16

Grouping - Using Series/Dictionaries as Key

```
In [92]: dfGrouped_oneKey_s = dfToGroup.groupby(key_plants)
print(f"The series 'plant' can be used to group the dfToGroup using dfGrouped_oneKey_s")
print(f"The command dfGrouped_oneKey_s.mean(numeric_only=True) returns:\n{dfGrouped_oneKey_s.mean(numeric_only=True)}\n")
```

The series 'plant' can be used to group the dfToGroup using dfGrouped_oneKey_s = dfToGroup.groupby(key_plants)

The command dfGrouped_oneKey_s.mean(numeric_only=True) returns:

	key2	data1	data2
Plant A	1.5	0.5	10.5
Plant B	1.5	2.5	12.5
Plant C	1.0	5.0	15.0

Grouping - Using Index Levels

- Grouping can be done based on the levels of a MultiIndex
 - This is done by specifying the level number or name
 - E.g., `df.groupby(level="level_name")`

Grouping - Using Index Levels

```
In [93]: print(f"df2 is: \n{df2}\n")
df2_multiID = df2.set_index(["state", "year"])
print(f"df2_multiID is: \n{df2_multiID}\n")
```



```
df2_multiID = df2.set_index(["state", "year"])
creates df2_multiID, a modified df2 with multiindex ["state", "year"]:
```

df2 is:

	year	state	pop	debt	income	debt_to_income
a	2000	Ohio	1500000	1.200000e+09	NaN	NaN
b	2001	Ohio	1700000	1.500000e+09	2000.0	750000.000000
c	2002	Ohio	3600000	1.700000e+09	NaN	NaN
d	2001	Nevada	2400000	5.000000e+08	3000.0	166666.666667
e	2002	Nevada	2900000	8.000000e+08	NaN	NaN
f	2003	Nevada	3200000	1.000000e+09	4000.0	250000.000000

The command

```
df2_multiID = df2.set_index(["state", "year"])
creates df2_multiID, a modified df2 with multiindex ["state", "year"]:
```

		pop	debt	income	debt_to_income
state	year				
Ohio	2000	1500000	1.200000e+09	NaN	NaN
	2001	1700000	1.500000e+09	2000.0	750000.000000
	2002	3600000	1.700000e+09	NaN	NaN
Nevada	2001	2400000	5.000000e+08	3000.0	166666.666667
	2002	2900000	8.000000e+08	NaN	NaN
	2003	3200000	1.000000e+09	4000.0	250000.000000

Grouping - Using Index Levels

```
In [94]: print(f"df2_multiID is: \n{df2_multiID}\n")
print(f"" The command
df2_multiID.groupby(level="state").count()
counts the number of not-NaN values in each column for each state: \n{df2_mu
```

df2_multiID is:

		pop	debt	income	debt_to_income
state	year				
Ohio	2000	1500000	1.200000e+09	NaN	NaN
	2001	1700000	1.500000e+09	2000.0	750000.000000
	2002	3600000	1.700000e+09	NaN	NaN
Nevada	2001	2400000	5.000000e+08	3000.0	166666.666667
	2002	2900000	8.000000e+08	NaN	NaN
	2003	3200000	1.000000e+09	4000.0	250000.000000

The command

```
df2_multiID.groupby(level="state").count()
counts the number of not-NaN values in each column for each state:
```

	pop	debt	income	debt_to_income
state				
Nevada	3	3	2	2
Ohio	3	3	1	1

Grouping - Using Index Levels

```
In [95]: print(f"df2_multiID is: \n{df2_multiID}\n")
print(f"" The command
```

```
df2_multiID.groupby(level="year").count()
counts the number of not-NaN values in each column for each year: \n{df2_mu1
```

df2_multiID is:

		pop	debt	income	debt_to_income
state	year				
Ohio	2000	1500000	1.200000e+09	NaN	NaN
	2001	1700000	1.500000e+09	2000.0	750000.000000
	2002	3600000	1.700000e+09	NaN	NaN
Nevada	2001	2400000	5.000000e+08	3000.0	166666.666667
	2002	2900000	8.000000e+08	NaN	NaN
	2003	3200000	1.000000e+09	4000.0	250000.000000

The command

```
df2_multiID.groupby(level="year").count()
counts the number of not-NaN values in each column for each year:
    pop  debt  income  debt_to_income
year
2000    1    1      0              0
2001    2    2      2              2
2002    2    2      0              0
2003    1    1      1              1
```

Grouping - Apply the Function `length` on the Index to Define the Key

Let's say that we want to group the data by the length of index names of the following dataset

```
In [96]: dfToGroupFun = pd.DataFrame(np.arange(25).reshape([5, 5]), columns=["a", "b", "c", "d", "e"],
                                     index=["Joe", "Jill", "Trey", "Steve", "Wanda"])
print(f"The dfToGroupFun DataFrame is: \n{dfToGroupFun}\n")
```

The dfToGroupFun DataFrame is:

	a	b	c	d	e
Joe	0	1	2	3	4
Jill	5	6	7	8	9
Trey	10	11	12	13	14
Steve	15	16	17	18	19
Wanda	20	21	22	23	24

Grouping - Apply the Function `length` on the Index to Define the Key

- Using `dfToGroupFun.groupby(len)` will compute the length of the names of the index create (three) datasets with the resulting groups
- By specifying `sum` in the aggregation, the resulting DataFrame will have the sum of each column for each group stacked vertically

```
In [97]: print(f""The command
dfToGroupFun.groupby(len).sum()
groups by the length of the index labels and performs the sum across the rows
```

The command

```
dfToGroupFun.groupby(len).sum()
```

groups by the length of the index labels and performs the sum across the rows of the resulting grouped dataFrames:

	a	b	c	d	e
3	0	1	2	3	4
4	15	17	19	21	23
5	35	37	39	41	43

Grouping - Apply a Lambda Function on a Column to Define the Key

- In some cases it is useful to group on a function of the value of a column
- Let's consider the familiar df2 DataFrame
 - We will group observations by brackets of population size

```
In [98]: print(f"df2 is:\n{df2}")
```

df2 is:

	year	state	pop	debt	income	debt_to_income
a	2000	Ohio	1500000	1.200000e+09	NaN	NaN
b	2001	Ohio	1700000	1.500000e+09	2000.0	750000.000000
c	2002	Ohio	3600000	1.700000e+09	NaN	NaN
d	2001	Nevada	2400000	5.000000e+08	3000.0	166666.666667
e	2002	Nevada	2900000	8.000000e+08	NaN	NaN
f	2003	Nevada	3200000	1.000000e+09	4000.0	250000.000000

Grouping - Apply a Lambda Function on a Column to Define the Key

- We define a lambda function that:
 - If the population is greater than 200 thousands, returns "Big Pop"
 - Otherwise, returns "Small Pop"
- Applying this function to the population column returns a series
 - Whose index matches the DataFrame one
 - Whose values are either 'Big Pop' or 'Small Pop'

Grouping - Apply a Lambda Function on a Column to Define the Key

```
In [99]: print(f""The command
df2['pop'].apply(lambda x: 'Big Pop' if x > 2000000 else 'Small Pop')
returns the series:\n{df2['pop'].apply(lambda x: 'Big Pop' if x > 2000000 else
```

The command
`df2['pop'].apply(lambda x: 'Big Pop' if x > 2000000 else 'Small Pop')`
 returns the series:

```
a    Small Pop
b    Small Pop
c     Big Pop
d     Big Pop
e     Big Pop
f     Big Pop
Name: pop, dtype: object
```

Grouping - Using Functions to Define Key

- Python functions offer the most flexible method for defining group
- These functions are executed on each index value by default
 - Can also be applied to the values of a column
- The function's return values determine the group names

Grouping - Use the Output of a Lambda Function to Group and Aggregate

- We can then use the series to group the dataset
- Finally we run the aggregating function sum on the grouped object

```
In [100... print(f""We can use this series to compute an aggregate function on the col
df2[['debt', 'income']].groupby(df2['pop'].apply(lambda x: 'Big Pop' if x >
```

We can use this series to compute an aggregate function on the columns 'debt' and 'income' by running:

```
df2[['debt', 'income']].groupby(df2['pop'].apply(lambda x: 'Big Pop' if x >
2000000 else 'Small Pop')).sum():
```

	debt	income
pop		
Big Pop	4.000000e+09	7000.0
Small Pop	2.700000e+09	2000.0

1. The command `df2['pop'].apply(lambda x: 'Big Pop' if x > 2000000 else 'Small Pop')` returns a series
 - The index is the same as the DataFrame, the values is 'Big Pop'/'Small Pop' depending on 'pop'
2. The series is used to **group the subset DataFrame** (`['debt', 'income']`)
3. `sum` the values of observations belonging to the **grouped subset DataFrame**

Data Aggregation in Pandas

- Aggregations are transformations yielding scalar values
 - They reduce the dimensionality of the data
 - Common aggregations include `mean` , `count` , `min` , `sum`

- Pandas optimizes these operations for efficiency
- Aggregations can be applied to a GroupBy object
 - Not limited to predefined methods/functions
 - Custom aggregation functions can be applied
 - Use `.aggregate()` or `.agg()` with custom functions

Some Aggregation Functions in Pandas

Function name	Description
any, all	Return True if any (one or more values) or all non-NA values are True
count	Number of non-NA values
first, last	First and last non-NA values
mean	Mean of non-NA values
median	Arithmetic median of non-NA values
min, max	Minimum and maximum of non-NA values
nth	Retrieve value that would appear at position n with the data in sorted order
ohlc	Compute four "open-high-low-close" statistics for time series-like data
prod	Product of non-NA values
quantile	Compute sample quantile
rank	Ordinal ranks of non-NA values, like calling Series.rank
size	Compute group sizes, returning result as a Series
sum	Sum of non-NA values
std, var	Sample standard deviation and variance

Data Aggregation in Pandas - Some Examples

```
In [101... print(f"dfGrouped_oneKey.size() returns the number of elements in each group")
print(f"dfGrouped_oneKey.first() returns the first element of each sorted group")
```

dfGrouped_oneKey.size() returns the number of elements in each group:

```
key1
a    3
b    2
dtype: int64
```

dfGrouped_oneKey.first() returns the first element of each sorted group:

```
key2  data1  data2
key1
a      1      0    10
b      1      4    14
```

Data Aggregation in Pandas - Using Custom Functions

```
In [102... def range_func(x):  
    return x.max() - x.min()  
print(f"dfToGroup is: \n{dfToGroup}\n")  
print(f"dfGrouped_oneKey.agg(range_func) returns: \n{dfGrouped_oneKey.agg(range_func)}\n")  
print(f"Lambda functions also work dfGrouped_oneKey.agg(lambda x: x.max() - x.min()) returns: \n{dfGrouped_oneKey.agg(lambda x: x.max() - x.min())}\n")
```

dfToGroup is:

	key1	key2	data1	data2
0	a	1	0	10
1	a	2	1	11
2	a	1	2	12
3	None	2	3	13
4	b	1	4	14
5	b	<NA>	5	15
6	None	1	6	16

dfGrouped_oneKey.agg(range_func) returns:

	key2	data1	data2
key1			
a	1	2	2
b	0	1	1

Lambda functions also work dfGrouped_oneKey.agg(lambda x: x.max() - x.min()) returns:

	key2	data1	data2
key1			
a	1	2	2
b	0	1	1

Using Multiple/Different Functions on Different Columns

- Multiple functions can be applied at once to different columns
 - Use a list specifying the functions
- If the functions that you want to apply are different for each column
 - Use a dictionary with the column names as keys and the functions to apply as values

Using Multiple/Different Functions on Different Columns

```
In [103... print(f"dfGrouped_oneKey.agg(['mean', 'median']) returns: \n{dfGrouped_oneKey.agg(['mean', 'median'])}\n")  
dict_col_fun = {"data1" : ["mean", "median"], "data2" : range_func}  
print(f"The column:functions dictionary dict_col_fun is: \n{dict_col_fun}\n")  
print(f"dfGrouped_oneKey.agg(dict_col_fun) returns: \n{dfGrouped_oneKey.agg(dict_col_fun)}\n")
```

dfGrouped_oneKey.agg(['mean', 'median']) returns:

	key2	data1		data2	
		mean	median	mean	median
key1					
a	1.333333	1.0	1.0	11.0	11.0
b	1.0	1.0	4.5	14.5	14.5

The column:functions dictionary dict_col_fun is:

```
{'data1': ['mean', 'median'], 'data2': <function range_func at 0x1129a1040>}
```

dfGrouped_oneKey.agg(dict_col_fun) returns:

	data1		data2
	mean	median	range_func
key1			
a	1.0	1.0	2
b	4.5	4.5	1

Data Aggregation in Pandas - `apply` Always The Most Flexible Option

- As we saw, `map` is a faster but limited version of `apply` on non-grouped data
 - The `aggregate` method is the fast but limited `apply` for grouped data
 - It is intended for scalar output
- The `apply` method is always the most flexible option
 - It can take any function returning a Pandas object (DataFrame or Series) or a scalar
 - Applies to all the groups
 - It will then attempt to combine the pieces
- Use `apply` when:
 - The output of the transformation is a DataFrame or Series
 - The transformation needs the data from multiple columns
 - The transformation needs additional parameters (other than the column data)

Data Aggregation Using `apply` - An Example

- We will now see an example of using `apply` to perform a group operation in a slightly less "toy" setting
- We will use the dataset "tips" (it comes with the seaborn library but we use a modified version)
 - Our aim is to uncover some features of the distribution of tips
 - We want to identify the highest n top and bottom tips in percentage by groups based on `day / time` combination

Data Aggregation Using `apply` - The Steps of the Example

1. Data Import
2. Definition of a function `compute_pctTip_topBot`
 - A. Computes the tip percentage of a bill
 - B. Select the highest n top and bottom tips in percentage
3. Application of the function by groups based on `day / time` combination
4. Refinements to improve readability (**CRUCIAL!**) of the table result
5. Saving the result to a file

Importing the `tips` Dataset

```
In [104... import os
#reads the file tips.csv and stores it into a df
tips = pd.read_csv('data/tips.csv')
print(f"Using pd.read_csv('data/tips.csv') to read tips.csv and store the da
tips
```

Using `pd.read_csv('data/tips.csv')` to read `tips.csv` and store the data in the variable `tips`:

```
Out[104...      total_bill  tip    sex  smoker  day  time  size
0      16.99   1.01  Female    No   Sun  Dinner    2
1      10.34   1.66   Male    No   Sun  Dinner    3
2      21.01   3.50   Male    No   Sun  Dinner    3
3      23.68   3.31   Male    No   Sun  Dinner    2
4      24.59   3.61  Female    No   Sun  Dinner    4
...         ...   ...     ...     ...   ...   ...   ...
239     29.03   5.92   Male    No   Sat  Dinner    3
240     27.18   2.00  Female   Yes   Sat  Dinner    2
241     22.67   2.00   Male   Yes   Sat  Dinner    2
242     17.82   1.75   Male    No   Sat  Dinner    2
243     18.78   3.00  Female    No  Thur  Dinner    2
```

244 rows × 7 columns

Some Info on the `tips` Dataset

The dataset `tips` contains information about tips in a restaurant:

- `total_bill`: Total bill (cost of the meal), including tax, in US dollars
- `tip`: Tip amount in US dollars
- `sex`: Gender of the person attending the table
- `smoker`: Indicates whether the table was in the smoking area or not

- `day` : Day of the week
- `time` : Meal (e.g., Lunch, Dinner)
- `size` : Number of people in the party

Grouping by `day / time` Combination

- The dataset provides the keys that we need for grouping in the columns `day` and `time`
- We need to define the function to apply to each group and extract the information that we need

Function Definition - `compute_pctTip_topBot`

```
In [105... # Defining the function compute_ratio_topBottom
def compute_pctTip_topBot(group, n):
    # Calculate the percentage of tip from the total bill and add it as a new column
    group['pctTip'] = 100 * group['tip'] / group['total_bill']
    # Select the top n rows with the highest 'pctTip' values
    top_n = group.nlargest(n, 'pctTip')
    # Select the bottom n rows with the lowest 'pctTip' values
    bottom_n = group.nsmallest(n, 'pctTip')
    # Concatenate the top and bottom rows into a single DataFrame
    #more on this soon, concatenate stacks the two dataframes one on top of the other
    result = pd.concat([top_n, bottom_n])
    # Return the combined DataFrame
    return result
```

Apply the Function `compute_pctTip_topBot` by `day / time` Groups

```
In [106... # Grouping the 'tips' DataFrame by 'day' and 'time', and applying the function
tips.groupby(['day', 'time']).apply(compute_pctTip_topBot, n=2, include_group_keys=True)
```

Out [106...

			total_bill	tip	sex	smoker	size	pctTip
day	time							
Fri	Dinner	93	16.32	4.30	Female	Yes	2	26.348039
		100	11.35	2.50	Female	Yes	2	22.026432
		90	28.97	3.00	Male	Yes	2	10.355540
		95	40.17	4.73	Male	Yes	4	11.774956
	Lunch	221	13.42	3.48	Female	Yes	2	25.931446
		222	8.58	1.92	Male	Yes	1	22.377622
		224	13.42	1.58	Male	Yes	2	11.773472
		225	16.27	2.50	Female	Yes	2	15.365704
Sat	Dinner	67	3.07	1.00	Female	Yes	1	32.573290
		232	11.61	3.39	Male	No	2	29.198966
		237	32.83	1.17	Male	Yes	2	3.563814
		102	44.30	2.50	Female	Yes	3	5.643341
Sun	Dinner	172	7.25	5.15	Male	Yes	2	71.034483
		178	9.60	4.00	Female	Yes	2	41.666667
		0	16.99	1.01	Female	No	2	5.944673
		187	30.46	2.00	Male	Yes	5	6.565988
Thur	Dinner	243	18.78	3.00	Female	No	2	15.974441
		243	18.78	3.00	Female	No	2	15.974441
	Lunch	149	7.51	2.00	Male	No	2	26.631158
		194	16.58	4.00	Male	Yes	2	24.125452
		146	18.64	1.36	Female	No	3	7.296137
		130	19.08	1.50	Male	No	2	7.861635

Refinements to the Table to Improve Readability

In [107...

```
# Map the weekday names to their corresponding numerical values
weekday_map = {'Mon': 1, 'Tue': 2, 'Wed': 3, 'Thur': 4, 'Fri': 5, 'Sat': 6,
# Map the meal times (e.g., 'Lunch', 'Dinner') to numerical values for sorting
time_map = {'Lunch': 1, 'Dinner': 2}
# Apply the weekday mapping to the 'day' column, creating a new column 'day_order'
tips['day_order'] = tips['day'].map(weekday_map)
# Apply the time mapping to the 'time' column, creating a new column 'time_order'
tips['time_order'] = tips['time'].map(time_map)
tips
```

Out[107...

	total_bill	tip	sex	smoker	day	time	size	day_order	time_order
0	16.99	1.01	Female	No	Sun	Dinner	2	7	2
1	10.34	1.66	Male	No	Sun	Dinner	3	7	2
2	21.01	3.50	Male	No	Sun	Dinner	3	7	2
3	23.68	3.31	Male	No	Sun	Dinner	2	7	2
4	24.59	3.61	Female	No	Sun	Dinner	4	7	2
...	...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	6	2
240	27.18	2.00	Female	Yes	Sat	Dinner	2	6	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2	6	2
242	17.82	1.75	Male	No	Sat	Dinner	2	6	2
243	18.78	3.00	Female	No	Thur	Dinner	2	4	2

244 rows × 9 columns

Refinements to the Table to Improve Readability

In [108...

```
# Group the DataFrame by 'day' and 'time', apply the compute_ratio_topBottom
# Sort the resulting DataFrame by 'day_order' and 'time_order'
df_pctTip_topBot_by_dayTime = tips.groupby(['day', 'time']).apply(compute_pctTip_topBot)
                              .sort_values(['day_order', 'time_order'])
df_pctTip_topBot_by_dayTime
```

Out [108...

			total_bill	tip	sex	smoker	size	day_order	time_order	
	day	time								
Thur	Lunch	149	7.51	2.00	Male	No	2	4	1	26.0
		194	16.58	4.00	Male	Yes	2	4	1	24.0
		146	18.64	1.36	Female	No	3	4	1	7.0
		130	19.08	1.50	Male	No	2	4	1	7.0
	Dinner	243	18.78	3.00	Female	No	2	4	2	15.0
		243	18.78	3.00	Female	No	2	4	2	15.0
	Fri	Lunch	221	13.42	3.48	Female	Yes	2	5	25.0
			222	8.58	1.92	Male	Yes	1	5	22.0
			224	13.42	1.58	Male	Yes	2	5	11.0
			225	16.27	2.50	Female	Yes	2	5	15.0
Fri	Dinner	93	16.32	4.30	Female	Yes	2	5	2	26.0
		100	11.35	2.50	Female	Yes	2	5	2	22.0
		90	28.97	3.00	Male	Yes	2	5	2	10.0
		95	40.17	4.73	Male	Yes	4	5	2	11.0
	Sat	Dinner	67	3.07	1.00	Female	Yes	1	6	32.0
			232	11.61	3.39	Male	No	2	6	29.0
			237	32.83	1.17	Male	Yes	2	6	3.0
			102	44.30	2.50	Female	Yes	3	6	5.0
Sun	Dinner	172	7.25	5.15	Male	Yes	2	7	2	71.0
		178	9.60	4.00	Female	Yes	2	7	2	41.0
		0	16.99	1.01	Female	No	2	7	2	5.0
		187	30.46	2.00	Male	Yes	5	7	2	6.0

Refinements to the Table to Improve Readability

In [109...

```
# Group the DataFrame by 'day' and 'time', apply the compute_ratio_topBottom
# Sort the resulting DataFrame by 'day_order' and 'time_order'
# Drop the columns 'day', 'day_order', 'time', and 'time_order' from the first
df_pctTip_topBot_by_dayTime = df_pctTip_topBot_by_dayTime.drop(columns=["day",
                                                                           "day_order",
                                                                           "time",
                                                                           "time_order"])
df_pctTip_topBot_by_dayTime = df_pctTip_topBot_by_dayTime.reset_index(level=[2], drop=True) #From the MultiIndex remove the 'day' and 'time'
df_pctTip_topBot_by_dayTime = df_pctTip_topBot_by_dayTime
```

Out [109...

		total_bill	tip	sex	smoker	size	pctTip
day	time						
Thur	Lunch	7.51	2.00	Male	No	2	26.631158
	Lunch	16.58	4.00	Male	Yes	2	24.125452
	Lunch	18.64	1.36	Female	No	3	7.296137
	Lunch	19.08	1.50	Male	No	2	7.861635
	Dinner	18.78	3.00	Female	No	2	15.974441
	Dinner	18.78	3.00	Female	No	2	15.974441
Fri	Lunch	13.42	3.48	Female	Yes	2	25.931446
	Lunch	8.58	1.92	Male	Yes	1	22.377622
	Lunch	13.42	1.58	Male	Yes	2	11.773472
	Lunch	16.27	2.50	Female	Yes	2	15.365704
	Dinner	16.32	4.30	Female	Yes	2	26.348039
	Dinner	11.35	2.50	Female	Yes	2	22.026432
	Dinner	28.97	3.00	Male	Yes	2	10.355540
	Dinner	40.17	4.73	Male	Yes	4	11.774956
Sat	Dinner	3.07	1.00	Female	Yes	1	32.573290
	Dinner	11.61	3.39	Male	No	2	29.198966
	Dinner	32.83	1.17	Male	Yes	2	3.563814
	Dinner	44.30	2.50	Female	Yes	3	5.643341
Sun	Dinner	7.25	5.15	Male	Yes	2	71.034483
	Dinner	9.60	4.00	Female	Yes	2	41.666667
	Dinner	16.99	1.01	Female	No	2	5.944673
	Dinner	30.46	2.00	Male	Yes	5	6.565988

Refinements to the Table to Improve Readability

In [110...

```
# For some day/time combinations, there are less than 4 observations leading
df_pctTip_topBot_by_dayTime = df_pctTip_topBot_by_dayTime.drop_duplicates()
df_pctTip_topBot_by_dayTime
```

Out[110]...

		total_bill	tip	sex	smoker	size	pctTip
day	time						
Thur	Lunch	7.51	2.00	Male	No	2	26.631158
	Lunch	16.58	4.00	Male	Yes	2	24.125452
	Lunch	18.64	1.36	Female	No	3	7.296137
	Lunch	19.08	1.50	Male	No	2	7.861635
	Dinner	18.78	3.00	Female	No	2	15.974441
Fri	Lunch	13.42	3.48	Female	Yes	2	25.931446
	Lunch	8.58	1.92	Male	Yes	1	22.377622
	Lunch	13.42	1.58	Male	Yes	2	11.773472
	Lunch	16.27	2.50	Female	Yes	2	15.365704
	Dinner	16.32	4.30	Female	Yes	2	26.348039
	Dinner	11.35	2.50	Female	Yes	2	22.026432
	Dinner	28.97	3.00	Male	Yes	2	10.355540
	Dinner	40.17	4.73	Male	Yes	4	11.774956
Sat	Dinner	3.07	1.00	Female	Yes	1	32.573290
	Dinner	11.61	3.39	Male	No	2	29.198966
	Dinner	32.83	1.17	Male	Yes	2	3.563814
	Dinner	44.30	2.50	Female	Yes	3	5.643341
Sun	Dinner	7.25	5.15	Male	Yes	2	71.034483
	Dinner	9.60	4.00	Female	Yes	2	41.666667
	Dinner	16.99	1.01	Female	No	2	5.944673
	Dinner	30.46	2.00	Male	Yes	5	6.565988

Saving the Table with Results to a File

- Now that we have the table with the results, we can save it to a file
 1. We will save it into markdown using the `to_markdown` method
 - The format used in jupyter notebooks
 2. We will save it to excel using the `to_excel` method,
 - A common format in the corporate world
- Both methods have several options to customize the output
 - Check the help for more details (e.g., `df_pctTip_topBot_by_dayTime.to_excel?`)

```
In [111... # Save the DataFrame to a Markdown file with GitHub-flavored markdown (requi
df_pctTip_topBot_by_dayTime.to_markdown(os.path.join('tables', 'df_pctTip_to
# Save the DataFrame to an Excel file with better formatting
df_pctTip_topBot_by_dayTime.to_excel(os.path.join('tables', 'df_pctTip_topBo
```

The Saved Table in Excel

	A	B	C	D	E	F	G	H	I	J	K	L
1	day	time	total_bill	tip	sex	smoker	size	pctTip				
2	Thur	Lunch	7,51	2	Male	No	2	26,63116				
3		Lunch	16,58	4	Male	Yes	2	24,12545				
4		Lunch	18,64	1,36	Female	No	3	7,296137				
5		Lunch	19,08	1,5	Male	No	2	7,861635				
6		Dinner	18,78	3	Female	No	2	15,97444				
7		Dinner	18,78	3	Female	No	2	15,97444				
8	Fri	Lunch	13,42	3,48	Female	Yes	2	25,93145				
9		Lunch	8,58	1,92	Male	Yes	1	22,37762				
10		Lunch	13,42	1,58	Male	Yes	2	11,77347				
11		Lunch	16,27	2,5	Female	Yes	2	15,3657				
12		Dinner	16,32	4,3	Female	Yes	2	26,34804				
13		Dinner	11,35	2,5	Female	Yes	2	22,02643				
14		Dinner	28,97	3	Male	Yes	2	10,35554				
15		Dinner	40,17	4,73	Male	Yes	4	11,77496				
16	Sat	Dinner	3,07	1	Female	Yes	1	32,57329				
17		Dinner	11,61	3,39	Male	No	2	29,19897				
18		Dinner	32,83	1,17	Male	Yes	2	3,563814				
19		Dinner	44,3	2,5	Female	Yes	3	5,643341				
20	Sun	Dinner	7,25	5,15	Male	Yes	2	71,03448				
21		Dinner	9,6	4	Female	Yes	2	41,66667				
22		Dinner	16,99	1,01	Female	No	2	5,944673				
23		Dinner	30,46	2	Male	Yes	5	6,565988				

Working with Multiple Data Objects in Pandas

Index Alignment for Arithmetic Operations

Arithmetic with Pandas - Automatic Index Alignment

- Pandas simplifies doing operations on multiple objects by accounting for their indexes
- When performing arithmetic operations and other data transformations, Pandas automatically aligns indexes
 - The resulting index is the union of the indexes of the two Pandas objects**
 - If index pairs don't match, missing values (`NaN`) are introduced

Arithmetic with Pandas - Automatic Index Alignment in Series

```
In [112... ser3 = pd.Series([7.3, -2.9, 3.4], index=["a", "c", "d"])
ser4 = pd.Series([-2.1, 2.7, -1.5], index=["c", "a", "e",])
print(f"ser3 Series: \n{ser3}\n")
```

```
print(f"ser4 Series: \n{ser4}\n")
print(f"Result of ser3 + ser4: \n{ser3 + ser4}")
```

```
ser3 Series:
a    7.3
c   -2.9
d    3.4
dtype: float64
```

```
ser4 Series:
c   -2.1
a    2.7
e   -1.5
dtype: float64
```

```
Result of ser3 + ser4:
a    10.0
c    -5.0
d     NaN
e     NaN
dtype: float64
```

Arithmetic with Pandas - Automatic Index Alignment in Series

- `ser3 + ser4` yields a series with indexes `a, c, d, e`
 - Overlapping indexes (`a` and `c`) are matched in the operation
 - Non-overlapping indexes (`d` and `e`) return `NaN`

Arithmetic with Pandas - Automatic Index Alignment in DataFrames

- The default behavior of Pandas is to **align series indexes with the dataframe columns**
 - The resulting index is the union of the indexes of the two Pandas objects
 - If index pairs don't match, missing values (`NaN`) are introduced (a union of the indexes)

Arithmetic with Pandas - Automatic Index Alignment in DataFrames

```
In [113... df4 = pd.DataFrame(np.arange(9.).reshape((3, 3)).transpose(), columns=["Colo
print(f"DataFrame df4:\n{df4}\n")
ser5 = pd.Series([10, 20, 30], index=["Colorado", "Texas", "Florida"])
print(f"Series ser5:\n{ser5}\n")
print(f"df4 + ser5 aligns the Series index with the DataFrame columns, broad
```



```
DataFrame df4:
      Colorado  Texas  Ohio
Colorado    0.0    3.0    6.0
Texas      1.0    4.0    7.0
Ohio       2.0    5.0    8.0
```

```
Series ser5:
Colorado    10
Texas       20
Florida     30
dtype: int64
```

df4 + ser5 aligns the Series index with the DataFrame columns, broadcasting down the rows:

```
      Colorado  Florida  Ohio  Texas
Colorado    10.0      NaN   NaN   23.0
Texas      11.0      NaN   NaN   24.0
Ohio       12.0      NaN   NaN   25.0
```

Arithmetic with Pandas - Automatic Index Alignment in DataFrames

- With Pandas arithmetic methods (e.g., `add`), use `axis=0` to align with the rows

```
In [114... print(f"DataFrame df4:\n{df4}\n")
print(f"Series ser5:\n{ser5}\n")
print(f"df4.add(ser5, axis=0) aligns the Series index with the DataFrame row
```

```
DataFrame df4:
      Colorado  Texas  Ohio
Colorado    0.0    3.0    6.0
Texas      1.0    4.0    7.0
Ohio       2.0    5.0    8.0
```

```
Series ser5:
Colorado    10
Texas       20
Florida     30
dtype: int64
```

df4.add(ser5, axis=0) aligns the Series index with the DataFrame rows, broadcasting across the columns:

```
      Colorado  Texas  Ohio
Colorado    10.0   13.0  16.0
Florida      NaN    NaN   NaN
Ohio         NaN    NaN   NaN
Texas       21.0   24.0  27.0
```

Panda's Arithmetic Methods

Method	Description
add	Method for addition
sub	Method for subtraction
div	Method for division
floordiv	Method for floor division
mul	Method for multiplication
mod	Method for modulo operation
pow	Method for exponential power

Exercises: Using Index Alignment in Arithmetic Methods

Exercise 1: Exchange Rate Conversion

You have a DataFrame containing sales data for a multinational company in different countries. There is a column called "Sales Amount (USD)" that represents the sales amount in U.S. dollars. You also have an exchange rate Series that provides the exchange rates for converting the sales amount to the local currency of each country. Create a new column called "Sales Amount (Local Currency)" that converts the sales amount from USD to the local currency using the provided exchange rates.

```
# Sample sales data DataFrame
sales_data = {'Country': ['USA', 'Germany', 'Japan', 'China',
                          'India'],
              'Sales Amount (USD)': [5000, 3500, 6000, 8000, 4500]}

sales_df = pd.DataFrame(sales_data)

# Sample exchange rate Series
exchange_rates = pd.Series(data=[1.0, 0.85, 110.0, 6.35, 75.0],
                           index=['USA', 'Germany', 'Japan',
                                  'China', 'India'],
                           name='Exchange Rate')

# Create a new column "Sales Amount (Local Currency)" in sales_df
by converting USD to local currency
```

Exercise 2: GDP Growth Rate Calculation

Calculate the annual GDP growth rate for a list of countries. You are provided with two DataFrames: `gdp_last_year` and `gdp_this_year`, each containing the GDP values for various countries in the respective years. Add a new column to `gdp_this_year` DataFrame named "GDP Growth Rate (%)" that calculates the percentage growth rate from the previous year.

```
# GDP data for the last year
gdp_last_year = pd.DataFrame({'Country': ['USA', 'China', 'India',
'Germany', 'Brazil'],
                                'GDP': [21.43, 14.34, 2.87, 3.86,
1.84]}) # Trillions USD

# GDP data for this year
gdp_this_year = pd.DataFrame({'Country': ['USA', 'China', 'India',
'Germany', 'Brazil'],
                                'GDP': [22.32, 15.42, 3.05, 3.95,
1.92]}) # Trillions USD

# Calculate GDP Growth Rate and add as a new column to
gdp_this_year DataFrame
```

Exercise 3: Portfolio Value Recalculation

Given a DataFrame `portfolio` containing the number of shares held in different companies and a Series `stock_prices` representing the current stock prices, calculate the total value of the portfolio for each company and add a new column "Total Value" to the `portfolio` DataFrame.

```
# Sample portfolio data
portfolio = pd.DataFrame({'Company': ['Apple', 'Google', 'Amazon',
'Tesla'],
                            'Shares': [50, 30, 20, 10]})

# Current stock prices
stock_prices = pd.Series(data=[150, 2800, 3300, 700],
                        index=['Apple', 'Google', 'Amazon',
'Tesla'],
                        name='Stock Price')

# Calculate Total Value of shares for each company and add as a new
column to portfolio DataFrame
```

Exercise 4: Inflation Adjustment

Adjust the prices of a list of products over the years for inflation. You are given a DataFrame `product_prices` with columns "Year", "Product", and "Price". You also have a Series `inflation_rate` that provides the annual inflation rate. Create a new column "Adjusted Price" that adjusts the prices for inflation to the current year.

```
# Sample product prices data
product_prices = pd.DataFrame({'Year': [2018, 2019, 2020, 2021,
2022],
                                'Product': ['Laptop', 'Laptop',
'Laptop', 'Laptop', 'Laptop'],
                                'Price': [1000, 1050, 1100, 1150,
1200]})
```

```
# Annual inflation rate
inflation_rate = pd.Series(data=[2.4, 1.8, 1.6, 2.2, 3.0],
                           index=[2018, 2019, 2020, 2021, 2022],
                           name='Inflation Rate (%)')
```

```
# Adjust product prices for inflation and add as a new column to
product_prices DataFrame
```

Exercise 5: Merging Sales Data with Currency Conversion Rates

Merge two DataFrames: `international_sales` containing sales amounts in USD and `currency_conversion` providing currency conversion rates. After merging, create a new column "Sales in Local Currency" to convert the sales amounts to local currencies.

```
# International sales data in USD
international_sales = pd.DataFrame({'Country': ['USA', 'UK',
'France', 'India', 'Japan'],
                                   'Sales Amount (USD)': [1000,
2000, 1500, 2500, 3000]})
```

```
# Currency conversion rates
currency_conversion = pd.DataFrame({'Country': ['USA', 'UK',
'France', 'India', 'Japan'],
                                   'Conversion Rate': [1.0, 0.75,
0.9, 75, 110]})
```

```
# Merge DataFrames and convert Sales Amount to local currency
```

Exercise 6: Stock Market Index Rebalancing

Given a DataFrame `stock_market_index` containing the composition of a stock market index and a Series `new_market_cap` providing the latest market capitalization for each stock, rebalance the index by updating the market capitalizations and calculating the new index weights.

```
# Stock market index composition
stock_market_index = pd.DataFrame({'Company': ['A', 'B', 'C', 'D',
'E'],
                                   'Market Cap (Billions USD)':
[500, 400, 300, 200, 100],
                                   'Index Weight (%)': [25, 20, 15,
10, 5]})
```

```
# Latest market capitalization for each company
```

```
new_market_cap = pd.Series
(
    data=[550, 420, 310, 190, 120],
    index=['A', 'B', 'C', 'D', 'E'],
    name='New Market Cap (Billions USD)')
```

Update market capitalizations and calculate new index weights

Exercise 7: Adjusting Income Data for Cost of Living

You have a DataFrame `income_data` containing average annual household incomes for various cities. Additionally, you are given a Series `cost_of_living_index` that provides the cost of living index for each city. Normally, Pandas aligns the Series index with the DataFrame columns, but you need to override this behavior to align the Series index with the DataFrame rows. This alignment is necessary to adjust the incomes for the cost of living in each city.

Task Description

- The DataFrame `income_data` has cities as rows and years as columns.
- The Series `cost_of_living_index` contains a cost of living index for each city.
- Adjust the household incomes in `income_data` by dividing each city's income by its corresponding cost of living index.

Sample Data and Code

```
import pandas as pd

# Average annual household income data (in USD)
income_data = pd.DataFrame({'2018': [60000, 80000, 55000],
                             '2019': [62000, 82000, 56000],
                             '2020': [64000, 85000, 58000]},
                             index=['New York', 'San Francisco',
                                    'Detroit'])

# Cost of living index for each city (higher values mean higher
# cost of living)
cost_of_living_index = pd.Series(data=[240, 280, 180],
                                  index=['New York', 'San
Francisco', 'Detroit'],
                                  name='Cost of Living Index')

# Adjust the incomes for cost of living
# Override the default behavior to align the Series index with the
DataFrame rows
```

I need you to provide me with code, thoroughly commented, to solve these exercises and display the results. Put all the code in a single python cell and separate the different exercises by printing the current exercise with separators

below you can find the solution to the first exercise that you can use as a template for its form

Working with Multiple Data Objects in Pandas

Data Concatenation and Merging

DataFrame Concatenation

- **Concatenating:** Stacks datasets by axis
 - `concat()` :
 - Stacks rows or binds columns (but for the columns merge is more flexible)
 - It is the method used when combining the result of groupby operations
 - Mostly used to stack rows using `axis=0` and ignoring indexes using `ignore_index=True`

DataFrame Concatenation in Pandas - `concat()` - `ignore_index=True`

```
In [115... df9 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df10 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
print(f"df9 is:\n{df9}\n\ndf10 is:\n{df10}\n")
print(f"The command pd.concat([df9, df10], axis=0, ignore_index=True) concat
```

df9 is:

	A	B
0	1	3
1	2	4

df10 is:

	A	B
0	5	7
1	6	8

The command `pd.concat([df9, df10], axis=0, ignore_index=True)` concatenates df9 and df10 along the rows:

	A	B
0	1	3
1	2	4
2	5	7
3	6	8

DataFrame Concatenation in Pandas - `concat()`

```
In [116... print(f"df9 is:\n{df9}\n\ndf10 is:\n{df10}\n")
print(f"The command pd.concat([df9, df10], axis=0) concatenates df9 and df10
```

df9 is:

	A	B
0	1	3
1	2	4

df10 is:

	A	B
0	5	7
1	6	8

The command `pd.concat([df9, df10], axis=0)` concatenates df9 and df10 along the rows keeping the index:

	A	B
0	1	3
1	2	4
0	5	7
1	6	8

DataFrame Concatenation in Pandas - `concat()` - Different Index Names Along Concatenation

```
In [117... df11 = pd.DataFrame({'A': [11, 12], 'B': [13, 14]})
df12 = pd.DataFrame({'C': [15, 16], 'D': [17, 18]})
print(f"df11 is:\n{df11}\n\ndf12 is:\n{df12}\n")
print(f"\"The command
pd.concat([df11, df12], axis=0, ignore_index=True) concatenates df11 and df12
{pd.concat([df11, df12], axis=0, ignore_index=True)}\"")
```

df11 is:

	A	B
0	11	13
1	12	14

df12 is:

	C	D
0	15	17
1	16	18

The command

`pd.concat([df11, df12], axis=0, ignore_index=True)` concatenates df11 and df12 (with different column names) along the rows:

	A	B	C	D
0	11.0	13.0	NaN	NaN
1	12.0	14.0	NaN	NaN
2	NaN	NaN	15.0	17.0
3	NaN	NaN	16.0	18.0

Merging

- **Merging:** Integrates dataframes columns based on ID/key
 - `merge()` is the most flexible function to merge

Syntax

```
pd.merge(df_a, df_b, on='key')
```

- `key` sets the column(s) storing the information identifying the same observations across dataframes

The Different Kinds of Merge

- Different kinds of merges can be performed using the parameter `how`

Option	Behavior
<code>how="inner"</code>	The output Dataframe keys are the intersection of the keys of both tables
<code>how="left"</code>	The output Dataframe keys are the ones from the left table
<code>how="right"</code>	The output Dataframe keys are the ones from the right table
<code>how="outer"</code>	The output Dataframe keys are the union of the key from both tables



The Left Merge

- We will focus on the **left merge** that is (allegedly) the most useful in data analysis
 - There is a main dataset (receiver) to augment with info from other datasets (donors)
 - E.g., An education economist investigating the impact of a policy on students' grades works with:
 - The core (receiver) dataset storing info on students' scores
 - Many other (donor) datasets providing info on timing of the policy and covariates
 - Socio-economic status, Age, Extracurricular activities, etc.

Doing a Left Merge in Pandas

- The command `pd.merge(df_a, df_b, on='key', how='left')` does a left merge of `df9` and `df10` on their 'key' columns:
 - **Observations in `df_a` (receiver) with matching 'key' in `df_b` (donor)**
 - Receive the information of the columns of `df_b`

- **Observations in df_a (receiver) without matching 'key' in df_b (donor)**
 - Are filled with NaN in the columns of df_b
- **Observations in df_b (donor) without matching 'key' in df_a (receiver)**
 - Are dropped

DataFrame Merging - merge()

```
In [118... df_WA = pd.DataFrame({'ID': ['jin', 'luca', 'mazhar', 'lynne', 'massimo', 'matthew'],
                        'weight': [60, 70, 66, 50, 65, 60],
                        'age': [11, 12, 12, 13, 12, 12]})
df_HA = pd.DataFrame({'ID': ['luca', 'lynne', 'matthew', 'jin', 'massimo', 'mazhar'],
                        'height': [170, 178, 164, np.nan, 180, 175],
                        'age': [12, 13, 12, 11, 12, 12]})
print(f"data_WA is: \n{df_WA}\n")
print(f"data_HA is: \n{df_HA}\n")
```

data_WA is:

	ID	weight	age
0	jin	60	11
1	luca	70	12
2	mazhar	66	12
3	lynne	50	13
4	massimo	65	12
5	matthew	60	12

data_HA is:

	ID	height	age
0	luca	170.0	12
1	lynne	178.0	13
2	matthew	164.0	12
3	jin	NaN	11
4	massimo	180.0	12
5	mazhar	175.0	12

DataFrame Merging - merge()

```
In [119... print(f"pd.merge(df_WA, df_HA, on='ID', how='left') merges df_HA and df_WA c
```

pd.merge(df_WA, df_HA, on='ID', how='left') merges df_HA and df_WA on the 'ID' column using a left join:

	ID	weight	age_x	height	age_y
0	jin	60	11	NaN	11
1	luca	70	12	170.0	12
2	mazhar	66	12	175.0	12
3	lynne	50	13	178.0	13
4	massimo	65	12	180.0	12
5	matthew	60	12	164.0	12

Join - A Simplified Index-Based Merge

- `join()` is a simplified left-merging based on index labels
 - `df_WA.join(df_HA)` left-joins `df_WA` (receiver) and `df_HA` (donor) on their indexes
 - The parameters `lsuffix` and `rsuffix` add the suffixes to overlapping column names
 - The default suffixes are `_x` and `_y`
- In this case the index do not represent the ID
 - This merge is **CORRUPTING THE INFORMATION IN THE DATA**
 - Any analysis based on this merge will be **WRONG**
 - We are just doing it for illustration purposes

Understanding DataFrame Merging in Pandas - `join()`

In [120... `print(f"df_WA.join(df_HA, how='left') joins df_WA and df_HA on the index using a left join")`
`print(f"df_WA.join(df_WA[['data2']], how='left') only adds the column of interest to df_WA")`

`df_WA.join(df_HA, how='left')` joins `df_WA` and `df_HA` on the index using a left join:

	ID_dfWA	weight	age_dfWA	ID_dfHA	height	age_dfHA
0	jin	60	11	luca	170.0	12
1	luca	70	12	lynne	178.0	13
2	mazhar	66	12	matthew	164.0	12
3	lynne	50	13	jin	NaN	11
4	massimo	65	12	massimo	180.0	12
5	matthew	60	12	mazhar	175.0	12

`df_WA.join(df_WA[['data2']], how='left')` only adds the column of interest from `df_WA`:

	ID	weight	age	height
0	jin	60	11	170.0
1	luca	70	12	178.0
2	mazhar	66	12	164.0
3	lynne	50	13	NaN
4	massimo	65	12	180.0
5	matthew	60	12	175.0

Strategy for Effective Merging - Check the Calls to Merge

Key Considerations for Merging DataFrames:

- **Potential for Errors:**
 - Merging is a delicate process that might lead to data corruption
 - Countless cases of scientific and business analysis invalidated by merging errors
 - A careful and methodical approach is crucial to ensure data integrity
- **Always triple check when merging**
 - You have just saw what might happen, and no warning nor error is raised

- If the AI is doing merges, check them **thoroughly**

Strategy for Effective Merging - Common Problems

- Often the results of a merge are not what you expect
 - Duplicates keys in one or both dataframes might lead to unexpected results
 - Multiple matches for the same key
 - Subtle differences in keys/ID can lead loss of information
 - Need to be of the same type (9 and "9" are different)
 - There might be small typing differences (e.g., "9" and " 9", "Padova" and "padova", "Nicolò" and "Nicolo")
 - In these cases the merge will return more NaN than it should

Data Reshaping

Different Shapes of Data - Wide and Long

Wide Format			
Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Long Format		
Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

Different Shapes of Data - Use Cases

- The shape of a given dataset depends on the preferences of the data provider
 - You might find data referring to the same observations in different shapes
 - It is then important to be able to reshape the data to merge them
- Apart from data ingestion, different shapes are more convenient in different tasks

- Depending on the function you want to perform, you need one shape or the other
- Pandas provides methods to reshape data
 - `stack()` and `unstack()` are the most common ones
 - `pivot()` and `melt()` are more advanced and flexible

Reshaping in Pandas - Some Resources

- Pandas provides a variety of methods to reshape data
 - `stack()` and `unstack()` are the most common ones
 - `pivot()` and `melt()` are more advanced and flexible
- We won't be able to look at this topic in the course
- Those of you that need this competence, can find excellent resources:
 - [Par 8.3 of Python for Data Analysis](#)
 - [The whole Ch.8 of Python for Data Analysis](#)
 - [Pandas documentation](#)

Additional Resources on Pandas

[Ch. 5-8 of Python for Data Analysis](#)

[Ch.3 of Python Data Science Handbook](#)

[Pandas documentation](#)

[Pandas quickstart tutorial](#)

Working with Multiple Data Objects in Pandas

Index Alignment for Arithmetic Operations

Arithmetic with Pandas - Automatic Index Alignment

- Pandas simplifies doing operations on multiple objects by accounting for their indexes
- When performing arithmetic operations and other data transformations, Pandas automatically aligns indexes
 - **The resulting index is the union of the indexes of the two Pandas objects**
 - If index pairs don't match, missing values (`NaN`) are introduced

Arithmetic with Pandas - Automatic Index Alignment in Series

```
In [121]: ser3 = pd.Series([7.3, -2.9, 3.4], index=["a", "c", "d"])
ser4 = pd.Series([-2.1, 2.7, -1.5], index=["c", "a", "e",])
print(f"ser3 Series: \n{ser3}\n")
```

```
print(f"ser4 Series: \n{ser4}\n")
print(f"Result of ser3 + ser4: \n{ser3 + ser4}")
```

```
ser3 Series:
a    7.3
c   -2.9
d    3.4
dtype: float64
```

```
ser4 Series:
c   -2.1
a    2.7
e   -1.5
dtype: float64
```

```
Result of ser3 + ser4:
a    10.0
c   -5.0
d     NaN
e     NaN
dtype: float64
```

Arithmetic with Pandas - Automatic Index Alignment in Series

- `ser3 + ser4` yields a series with indexes `a, c, d, e`
 - Overlapping indexes (`a` and `c`) are matched in the operation
 - Non-overlapping indexes (`d` and `e`) return `NaN`

Arithmetic with Pandas - Automatic Index Alignment in DataFrames

- The default behavior of Pandas is to **align series indexes with the dataframe columns**
 - The resulting index is the union of the indexes of the two Pandas objects
 - If index pairs don't match, missing values (`NaN`) are introduced (a union of the indexes)

Arithmetic with Pandas - Automatic Index Alignment in DataFrames

In [122...

```
df4 = pd.DataFrame(np.arange(9.).reshape((3, 3)).transpose(), columns=["Colo", "Tex", "Fla"])
print(f"DataFrame df4:\n{df4}\n")
ser5 = pd.Series([10, 20, 30], index=["Colorado", "Texas", "Florida"])
print(f"Series ser5:\n{ser5}\n")
print(f"df4 + ser5 aligns the Series index with the DataFrame columns, broad"
```

```
DataFrame df4:
      Colorado  Texas  Ohio
Colorado    0.0    3.0    6.0
Texas      1.0    4.0    7.0
Ohio       2.0    5.0    8.0
```

```
Series ser5:
Colorado    10
Texas       20
Florida     30
dtype: int64
```

df4 + ser5 aligns the Series index with the DataFrame columns, broadcasting down the rows:

```
      Colorado  Florida  Ohio  Texas
Colorado    10.0      NaN   NaN   23.0
Texas      11.0      NaN   NaN   24.0
Ohio       12.0      NaN   NaN   25.0
```

Arithmetic with Pandas - Automatic Index Alignment in DataFrames

- With Pandas arithmetic methods (e.g., `add`), use `axis=0` to align with the rows

```
In [123... print(f"DataFrame df4:\n{df4}\n")
print(f"Series ser5:\n{ser5}\n")
print(f"df4.add(ser5, axis=0) aligns the Series index with the DataFrame row
```

```
DataFrame df4:
      Colorado  Texas  Ohio
Colorado    0.0    3.0    6.0
Texas      1.0    4.0    7.0
Ohio       2.0    5.0    8.0
```

```
Series ser5:
Colorado    10
Texas       20
Florida     30
dtype: int64
```

df4.add(ser5, axis=0) aligns the Series index with the DataFrame rows, broadcasting across the columns:

```
      Colorado  Texas  Ohio
Colorado    10.0   13.0  16.0
Florida      NaN    NaN   NaN
Ohio         NaN    NaN   NaN
Texas       21.0   24.0  27.0
```

Panda's Arithmetic Methods

Method	Description
add	Method for addition
sub	Method for subtraction
div	Method for division
floordiv	Method for floor division
mul	Method for multiplication
mod	Method for modulo operation
pow	Method for exponential power

Exercises: Using Index Alignment in Arithmetic Methods

Exercise 1: Exchange Rate Conversion

You have a DataFrame containing sales data for a multinational company in different countries. There is a column called "Sales Amount (USD)" that represents the sales amount in U.S. dollars. You also have an exchange rate Series that provides the exchange rates for converting the sales amount to the local currency of each country. Create a new column called "Sales Amount (Local Currency)" that converts the sales amount from USD to the local currency using the provided exchange rates.

```
# Sample sales data DataFrame
sales_data = {'Country': ['USA', 'Germany', 'Japan', 'China',
                          'India'],
              'Sales Amount (USD)': [5000, 3500, 6000, 8000, 4500]}

sales_df = pd.DataFrame(sales_data)

# Sample exchange rate Series
exchange_rates = pd.Series(data=[1.0, 0.85, 110.0, 6.35, 75.0],
                           index=['USA', 'Germany', 'Japan',
                                  'China', 'India'],
                           name='Exchange Rate')

# Create a new column "Sales Amount (Local Currency)" in sales_df
by converting USD to local currency
```

Exercise 2: GDP Growth Rate Calculation

Calculate the annual GDP growth rate for a list of countries. You are provided with two DataFrames: `gdp_last_year` and `gdp_this_year`, each containing the GDP values for various countries in the respective years. Add a new column to `gdp_this_year` DataFrame named "GDP Growth Rate (%)" that calculates the percentage growth rate from the previous year.

```

# GDP data for the last year
gdp_last_year = pd.DataFrame({'Country': ['USA', 'China', 'India',
'Germany', 'Brazil'],
                                'GDP': [21.43, 14.34, 2.87, 3.86,
1.84]}) # Trillions USD

# GDP data for this year
gdp_this_year = pd.DataFrame({'Country': ['USA', 'China', 'India',
'Germany', 'Brazil'],
                                'GDP': [22.32, 15.42, 3.05, 3.95,
1.92]}) # Trillions USD

# Calculate GDP Growth Rate and add as a new column to
gdp_this_year DataFrame

```

Exercise 3: Portfolio Value Recalculation

Given a DataFrame `portfolio` containing the number of shares held in different companies and a Series `stock_prices` representing the current stock prices, calculate the total value of the portfolio for each company and add a new column "Total Value" to the `portfolio` DataFrame.

```

# Sample portfolio data
portfolio = pd.DataFrame({'Company': ['Apple', 'Google', 'Amazon',
'Tesla'],
                            'Shares': [50, 30, 20, 10]})

# Current stock prices
stock_prices = pd.Series(data=[150, 2800, 3300, 700],
                        index=['Apple', 'Google', 'Amazon',
'Tesla'],
                        name='Stock Price')

# Calculate Total Value of shares for each company and add as a new
column to portfolio DataFrame

```

Exercise 4: Inflation Adjustment

Adjust the prices of a list of products over the years for inflation. You are given a DataFrame `product_prices` with columns "Year", "Product", and "Price". You also have a Series `inflation_rate` that provides the annual inflation rate. Create a new column "Adjusted Price" that adjusts the prices for inflation to the current year.

```

# Sample product prices data
product_prices = pd.DataFrame({'Year': [2018, 2019, 2020, 2021,
2022],
                                'Product': ['Laptop', 'Laptop',
'Laptop', 'Laptop', 'Laptop'],
                                'Price': [1000, 1050, 1100, 1150,
1200]})

```



```
# Annual inflation rate
inflation_rate = pd.Series(data=[2.4, 1.8, 1.6, 2.2, 3.0],
                           index=[2018, 2019, 2020, 2021, 2022],
                           name='Inflation Rate (%)')
```

```
# Adjust product prices for inflation and add as a new column to
product_prices DataFrame
```

Exercise 5: Merging Sales Data with Currency Conversion Rates

Merge two DataFrames: `international_sales` containing sales amounts in USD and `currency_conversion` providing currency conversion rates. After merging, create a new column "Sales in Local Currency" to convert the sales amounts to local currencies.

```
# International sales data in USD
international_sales = pd.DataFrame({'Country': ['USA', 'UK',
'France', 'India', 'Japan'],
                                   'Sales Amount (USD)': [1000,
2000, 1500, 2500, 3000]})
```

```
# Currency conversion rates
currency_conversion = pd.DataFrame({'Country': ['USA', 'UK',
'France', 'India', 'Japan'],
                                   'Conversion Rate': [1.0, 0.75,
0.9, 75, 110]})
```

```
# Merge DataFrames and convert Sales Amount to local currency
```

Exercise 6: Stock Market Index Rebalancing

Given a DataFrame `stock_market_index` containing the composition of a stock market index and a Series `new_market_cap` providing the latest market capitalization for each stock, rebalance the index by updating the market capitalizations and calculating the new index weights.

```
# Stock market index composition
stock_market_index = pd.DataFrame({'Company': ['A', 'B', 'C', 'D',
'E'],
                                   'Market Cap (Billions USD)':
[500, 400, 300, 200, 100],
                                   'Index Weight (%)': [25, 20, 15,
10, 5]})
```

```
# Latest market capitalization for each company
```

```
new_market_cap = pd.Series
(
    data=[550, 420, 310, 190, 120],
    index=['A', 'B', 'C', 'D', 'E'],
    name='New Market Cap (Billions USD)')
```

Update market capitalizations and calculate new index weights

Exercise 7: Adjusting Income Data for Cost of Living

You have a DataFrame `income_data` containing average annual household incomes for various cities. Additionally, you are given a Series `cost_of_living_index` that provides the cost of living index for each city. Normally, Pandas aligns the Series index with the DataFrame columns, but you need to override this behavior to align the Series index with the DataFrame rows. This alignment is necessary to adjust the incomes for the cost of living in each city.

Task Description

- The DataFrame `income_data` has cities as rows and years as columns.
- The Series `cost_of_living_index` contains a cost of living index for each city.
- Adjust the household incomes in `income_data` by dividing each city's income by its corresponding cost of living index.

Sample Data and Code

```
import pandas as pd

# Average annual household income data (in USD)
income_data = pd.DataFrame({'2018': [60000, 80000, 55000],
                             '2019': [62000, 82000, 56000],
                             '2020': [64000, 85000, 58000]},
                             index=['New York', 'San Francisco',
                                    'Detroit'])

# Cost of living index for each city (higher values mean higher
# cost of living)
cost_of_living_index = pd.Series(data=[240, 280, 180],
                                  index=['New York', 'San
Francisco', 'Detroit'],
                                  name='Cost of Living Index')

# Adjust the incomes for cost of living
# Override the default behavior to align the Series index with the
DataFrame rows
```

I need you to provide me with code, thoroughly commented, to solve these exercises and display the results. Put all the code in a single python cell and separate the different exercises by printing the current exercise with separators

below you can find the solution to the first exercise that you can use as a template for its form

Merging

Strategy for Effective Merging - The Checks to Do When Merging

- **Check the keys/ID of each dataset for duplicates**
- **Identify Non-Matching keys/ID Before Merging:**
 - Identify the keys in the receiver dataframe that do not have a match in the donor dataframe
 - For these obs the donated columns will be `NaN` in the merged dataframe
 - Better to use indexes for this check
 - Allows to use the `.index.get_level_values` method
 - For multiindex DataFrames creates automatically the combinations of indexes

Strategy for Effective Merging - The Checks to Do When Merging

- **Prefer Adding Columns Individually:**
 - Enhances code readability and manageability
 - Adding one column at a time makes it easier track `NaN` values and perform the checks below:
 1. **Check the NaN in the column to be received**
 - The column stored in the donor dataframe
 - If there are `NaN` values they will pass to the receiver dataframe
 2. **Check the NaN in the column received**
 - The column stored in the receiver dataframe (only available after the merge)
 - The number of `NaN` should be equal to the **sum** of:
 - The number of `NaN` in the donor dataframe plus
 - the number of indexes/keys/ID in the receiver not in the donor

Strategy for Effective Merging - The Datasets

- Let's create two datasets to illustrate the checks to do when merging

In [124...

```
df_WAmod = df_WA[df_WA['ID'] != 'luca'].copy()
print(f"The command df_WAmod = df_WAmod[df_WAmod['ID'] != 'luca'].copy() cre
df_HAmod = df_HA[df_HA['ID'] != 'lynne'].copy()
print(f"The command df_HAmod = df_HA[df_HA['ID'] != 'lynne'].copy() creates
```

The command `df_WAmod = df_WA[df_WA['ID'] != 'luca'].copy()` creates a new DataFrame from `df_WA` by removing the row with ID 'luca':

	ID	weight	age
0	jin	60	11
2	mazhar	66	12
3	lynne	50	13
4	massimo	65	12
5	matthew	60	12

The command `df_HAmod = df_HA[df_HA['ID'] != 'lynne'].copy()` creates a new DataFrame from `df_HA` by removing the row with ID 'lynne':

	ID	height	age
0	luca	170.0	12
2	matthew	164.0	12
3	jin	NaN	11
4	massimo	180.0	12
5	mazhar	175.0	12

Strategy for Effective Merging - Reindexing

- Reindexing allows to use the method `.index.get_level_values` to get the keys
 - Not needed in our example with just one column for key
 - Convenient when there are multiple columns for the key

Strategy for Effective Merging - Reindexing

```
In [125... df_WAmod.set_index('ID', inplace=True)
print(f"The command df_WAmod.set_index('ID', inplace=True) sets the index of
df_HAmod.set_index('ID', inplace=True)
print(f"The command df_HAmod.set_index('ID', inplace=True) sets the index of
```

The command `df_WAmod.set_index('ID', inplace=True)` sets the index of `df_WAmod` to 'ID':

	weight	age
ID		
jin	60	11
mazhar	66	12
lynne	50	13
massimo	65	12
matthew	60	12

The command `df_HAmod.set_index('ID', inplace=True)` sets the index of `df_HAmod` to 'ID':

	height	age
ID		
luca	170.0	12
matthew	164.0	12
jin	NaN	11
massimo	180.0	12
mazhar	175.0	12

Strategy for Effective Merging - Identifying Non-Matching IDs/indexes/keys

- In a MultiIndex, the `get_level_values()` method returns combinations of the index values at the specified level
 - This uniquely identifies each observation
 - Using a column is fine for a single key
 - With multiple IDs/indexes/keys it is better to use a MultiIndex

Strategy for Effective Merging - Identifying Non-Matching IDs/indexes/keys

```
In [126... idsReceiver_notIn_donor = set(df_WAmod.index.get_level_values('ID')) - set(c
print(f""The command
idsReceiver_notIn_donor = set(df_WAmod.index.get_level_values('ID')) - set(c
identifies unique IDs in df_WAmod (receiver) but not in df_HAmod (donor):\n
{idsReceiver_notIn_donor}""")
```

The command

```
idsReceiver_notIn_donor = set(df_WAmod.index.get_level_values('ID')) - set(d
f_HAmod.index.get_level_values('ID'))
identifies unique IDs in df_WAmod (receiver) but not in df_HAmod (donor):
```

```
{'lynne'}
```

- This check is in line with our expectations
 - No NaN arising due to typing differences or oddities due to duplicates
- It identifies the IDs that will result in NaN during the merge
 - These will be added to the ones already in the donor column

Strategy for Effective Merging - Merge the DataFrame one Column at a Time

- Since we set the key column as index we can directly use `join`

```
In [127... df_WAmodJoined = df_WAmod.join(df_HAmod['height'])
print(f""The command
df_WAmodJoined = df_WAmod.join(df_HAmod['height'])
adds the column 'height' from df_HAmod (donor) to df_WAmod (receiver) on the
df_WAmodJoined
```

The command

```
df_WAmodJoined = df_WAmod.join(df_HAmod['height'])
adds the column 'height' from df_HAmod (donor) to df_WAmod (receiver) on the
index:
```

Out [127...

	weight	age	height
ID			
jin	60	11	NaN
mazhar	66	12	175.0
lynne	50	13	NaN
massimo	65	12	180.0
matthew	60	12	164.0

Strategy for Effective Merging - Compare the NaN Pre/Post Merge

- We want to check the indexes of the NaN in:
 1. The column **to be received** (in the donor DataFrame)
 2. The column **effectively received** after merge (in the receiver DataFrame)

```
In [128... heightNan_idsDonor = set(df_HAmod[df_HAmod['height'].isna()].index.get_level_
print(f""The command
heightNan_idsDonor = set(df_HAmod[df_HAmod['height'].isna()].index.get_level_
stores the index of the NaN values in the 'height' column of the donor: \n{
heightNan_idsReceived = set(df_WAmodJoined[df_WAmodJoined['height'].isna()].
print(f""The command
heightNan_idsReceived = set(df_WAmodJoined[df_WAmodJoined['height'].isna()].
stores the indexes of the NaN values in the 'height' column received: \n{hei
```

The command

```
heightNan_idsDonor = set(df_HAmod[df_HAmod['height'].isna()].index.get_level_
_values('ID'))
stores the index of the NaN values in the 'height' column of the donor:
{'jin'}
```

The command

```
heightNan_idsReceived = set(df_WAmodJoined[df_WAmodJoined['height'].isna()].
index.get_level_values('ID'))
stores the indexes of the NaN values in the 'height' column received:
{'jin', 'lynne'}
```

Strategy for Effective Merging - NaN Pre/Post Merge vs Non-Matching IDs/indexes/keys

- We then compare the indexes with NaN in the received column (post merge) with the **union** of:
 1. The index of NaN in the column to be received (in the donor DataFrame)
 2. The non-matching IDs/indexes/keys

```
In [129... print(f""The command
heightNan_idsReceived - (heightNan_idsDonor.union(idsReceiver_notIn_donor))
```

returns the IDs that are NaN in the receiver but not in the donor nor result

The command

```
heightNan_idsReceived - (heightNan_idsDonor.union(idsReceiver_notIn_donor))
```

returns the IDs that are NaN in the receiver but not in the donor nor result
ing from an IDs/indexes/keys mismatch:
set()

- In our example the result is an empty set
 - The merge went through as expected